Workshop!

Questions & Hints

Software Project Management

Three-point estimate technique

Three-point estimate technique

- The three-point estimate technique combines optimistic, pessimistic, and most likely estimates to determine a realistic timeframe for a project:

1. **Optimistic estimate (O)**
2. **Most likely estimate (M)**
3. **Pessimistic estimate (P)**

Three-point estimate technique

- TechSolutions Inc. has been contracted to develop a mobile app for a local fitness center. The project manager uses the three-point estimate technique to assess the timeline for the development phase. The most likely estimate for completing the app development is 8 weeks, but unforeseen technical challenges could extend it to 12 weeks. Conversely, if everything goes smoothly, the development might be finished in just 6 weeks. Based on the three-point estimate:

1. what is the most realistic timeframe TechSolutions Inc. should communicate to the fitness center for completing the mobile app development?

2. What proactive measures can TechSolutions Inc. implement to mitigate the risk of unforeseen technical challenges and ensure timely completion of the mobile app development project?

Three-point estimate technique

- Solution: The three-point estimate technique combines optimistic, pessimistic, and most likely estimates to determine a realistic timeframe for a project.
- *Given:*
- *Optimistic estimate (O) = 6 weeks*
- *Most likely estimate (M) = 8 weeks*
- *Pessimistic estimate (P) = 12 weeks*

Three-point estimate technique

- *Expected Timeframe: (O + 4M + P) / 6*
- ***Weighted Average (emphasizing Most Likely)**: (O + 4M + P) / 6*
- *= (6 + 32 + 12) / 6*
- *= 50 / 6*
- *= **8.33 weeks***
- *It's important to note that this is an estimate and actual times may vary based on project execution. It's also good practice to include a buffer in the timeline to account for any unforeseen circumstances.*

Software Project Management

Cost Analysis and Resource Management in Software Projects

Cost Analysis and Resource Management in Software Projects

- Sunshine Software is developing a new cloud-based project management tool. Initial estimates suggest the project will require 8 developers for a period of 9 months. The average developer salary at Sunshine Software is $120,000 per year (including benefits). There are also additional overhead costs estimated at 25% of the total salaries.

1. What is the total cost of developer salaries and overhead for the project?

2. What strategies can Sunshine Software implement to optimize resource utilization and minimize costs during the development of the cloud-based project management tool?

Cost Analysis and Resource Management in Software Projects

Solution!

- To calculate the total projected cost for the development team, we need to consider both the **salaries** and the **overhead costs**.

Given:

- *Number of developers: 8*
- *Project duration: 9 months*
- *Average developer salary: $120,000 per year*

Cost Analysis and Resource Management in Software Projects

Solution Cont.!

- **Step 1**: **Monthly Developer Salary** = [Annual Salary / 12)]
  =[$120,000 × 12]
  =$10,000

- **Step 2: Total Salary:**

Total Salary = [Salary × # of Developer × 9 months]
=[$10,000 × 8 × 9]
=$720,000

Cost Analysis and Resource Management in Software Projects

Solution Cont.!

- **Step 3: Overhead Costs**:

Overhead cost = [Total Salary × Overhead Rate]
= $720,000 × 0.25
=$180,000

- **Step 4: Total Projected Cost:**

Total Projected cost = [Total Salary cost + Overhead Cost]
= $720,000 + $180,000
=$900,000

Therefore, the total cost of developer salaries and overhead for the project is $900,000.

Software Metrics and Project Estimation

Lines of code (LOC)-based estimation technique

Lines of code (LOC)-based estimation technique

- You are leading a project for a software development company tasked with creating a mobile application for a local restaurant. This application will allow customers to browse the menu, place orders for pickup or delivery, make reservations, and provide feedback on their dining experience. The total estimated lines of code (LOC) for the application is 9,500.

1. As part of your project planning, you need to estimate the effort and cost required to build this software using a lines of code (LOC)-based estimation technique. Your organization's productivity rate is 500 LOC per person-month, and the labor rate, including overhead costs, is $6,000 per person-month.
2. Discuss potential factors or risks that could impact the accuracy of the lines of code (LOC)-based estimation technique in predicting the effort and cost of software development projects. How can project managers mitigate these factors to improve estimation accuracy?

Lines of code (LOC)-based estimation technique

- Solution:
- To estimate the **effort** and **cost** required to build the mobile application for the local restaurant, we'll use the given lines of code (LOC)-based estimation technique.

Given:

- *Total estimated lines of code (LOC):* ***9,500***
- *Productivity rate:* ***500 LOC*** *per person-month*
- *Labor rate, including overhead costs:* ***$6,000*** *per person-month*

Lines of code (LOC)-based estimation technique

Solution!

- Step 1: Calculate Effort Required:

Effort (in person-months) = [Total LOC / Productivity rate]

Effort = 9,500 LOC / 500 LOC per person-month

= **19 person-months**

- Step 2: Calculate Cost of Development:

Cost of development = Effort × Labor rate

Cost = 19 person-months × $6,000 per person-month

= **$114,000**

Therefore, the effort required to build the software is approximately **19 person-months**, and the cost of development is **$114,000**

Reliability Engineering

Software Reliability Calculation

Software Reliability Calculation

- A simple measure of reliability is **mean-time-between-failure (MTBF)**, where

MTBF = MTTF + MTTR

- The acronyms MTTF and MTTR are mean-time-to-failure and mean-time-to-repair, respectively.

- **Software availability** is the probability that a program is operating according to requirements at a given point in time and is defined as

- **Availability = [MTTF/(MTTF + MTTR)] x 100%**

Software Reliability Calculation

Scenario:
GlobalBank operates an online transaction processing system that supports millions of customers worldwide. The system handles ATM withdrawals, credit card transactions, and mobile payments. Due to its critical financial nature, regulators require banks to maintain a minimum availability of **99.5%**.
The system's engineers conducted an operational reliability study and observed the following:
The average **Mean Time To Failure (MTTF)** is **400 hours** (after 400 hours of continuous operation, the system is likely to fail).
The average **Mean Time To Repair (MTTR)** is **10 hours** (time taken to restore the system after a failure).
The bank is considering investing in additional **redundancy** measures (active–passive replication), which could reduce MTTR to **2 hours**, though at a significant infrastructure cost.

Software Reliability Calculation

Questions:

(a) Using the given MTTF and MTTR values, calculate the system's current **availability percentage**.

(b) Based on your calculation, evaluate whether the system meets the regulator's minimum requirement of 99.5% availability.

(c) If redundancy reduces MTTR to 2 hours, recalculate the system availability.

Would this investment justify the cost from a regulatory and business continuity perspective? Provide a critical discussion.

Software Reliability Calculation

Solutions:

(a) Availability Calculation:

$$Availability = \frac{MTTF}{MTTF + MTTR}$$

$$Availability = \frac{400}{400 + 10} = \frac{400}{410} \approx 0.9756 = 97.56\%$$

Software Reliability Calculation

Solutions:

(b) Evaluation:
The required availability is 99.5%. The system achieves only 97.56% → **Fails to meet regulatory requirements**.

(c) With redundancy (MTTR = 2 hours):

$$Availability = \frac{400}{400 + 2} = \frac{400}{402} \approx 0.995 = 99.5\%$$

This exactly meets the regulator's threshold.

Tutorial - 05

Case Study Scenario: ABC Software Solutions is a leading provider of cloud-based accounting software for small businesses. One of their flagship products, "FinanceMaster," is widely used by entrepreneurs and business owners to manage their financial transactions, generate reports, and streamline accounting processes.

- The reliability and availability of FinanceMaster are critical factors in ensuring customer satisfaction and business success. To assess the software's performance, ABC Software Solutions utilizes the mean-time-between-failure (MTBF) metric, which is calculated as the sum of the mean-time-to-failure (MTTF) and the mean-time-to-repair (MTTR).
- MTTF represents the average time between software failures, while MTTR denotes the average time required to repair the software after a failure occurs.
- Given the importance of software availability in maintaining customer trust and confidence, ABC Software Solutions also measures the software's availability, defined as the probability that FinanceMaster is operating according to requirements at any given point in time. This availability metric is crucial for ensuring uninterrupted access to critical financial data and functionalities.
- Let's consider a scenario where FinanceMaster has the following reliability metrics:
- Mean-Time-to-Failure (MTTF) = 200 hours
- Mean-Time-to-Repair (MTTR) = 20 hours
- Your task is to calculate the availability of FinanceMaster and analyze the implications of this metric for ABC Software Solutions' business operations and customer satisfaction. Additionally, discuss potential strategies for improving software availability and mitigating the impact of system failures on users.

Workshop!

Questions & Hints

Summary of Coverage

- Software quality & Software standards
- Reviews/Inspections (static) & Testing (Dynamic)
- Reliability & Security(14) Engineering

- *Software Project Management*

Moderated Exam

Applications Development Theory 4

- **Subject:** APT470S
- **Date:** Wednesday 05-Nov. @14:00 (FT/PT)
- **Venue:** M.P.H (Written)

- **Sick Test:** 19 Nov 2025 @ 09:00

Review Techniques

What Are Reviews?

- A meeting conducted by technical people for technical people
- A technical assessment of a work product created during the software engineering process
- A software quality assurance mechanism

What Reviews Are Not?

- A project summary or progress assessment
- A meeting intended solely to impart information
- A mechanism for political or personal reprisal!

What Do We Look For?

- Errors and defects
  - *Error*—a quality problem found *before* the software is released to end users
  - *Defect*—a quality problem found only *after* the software has been released to end-users

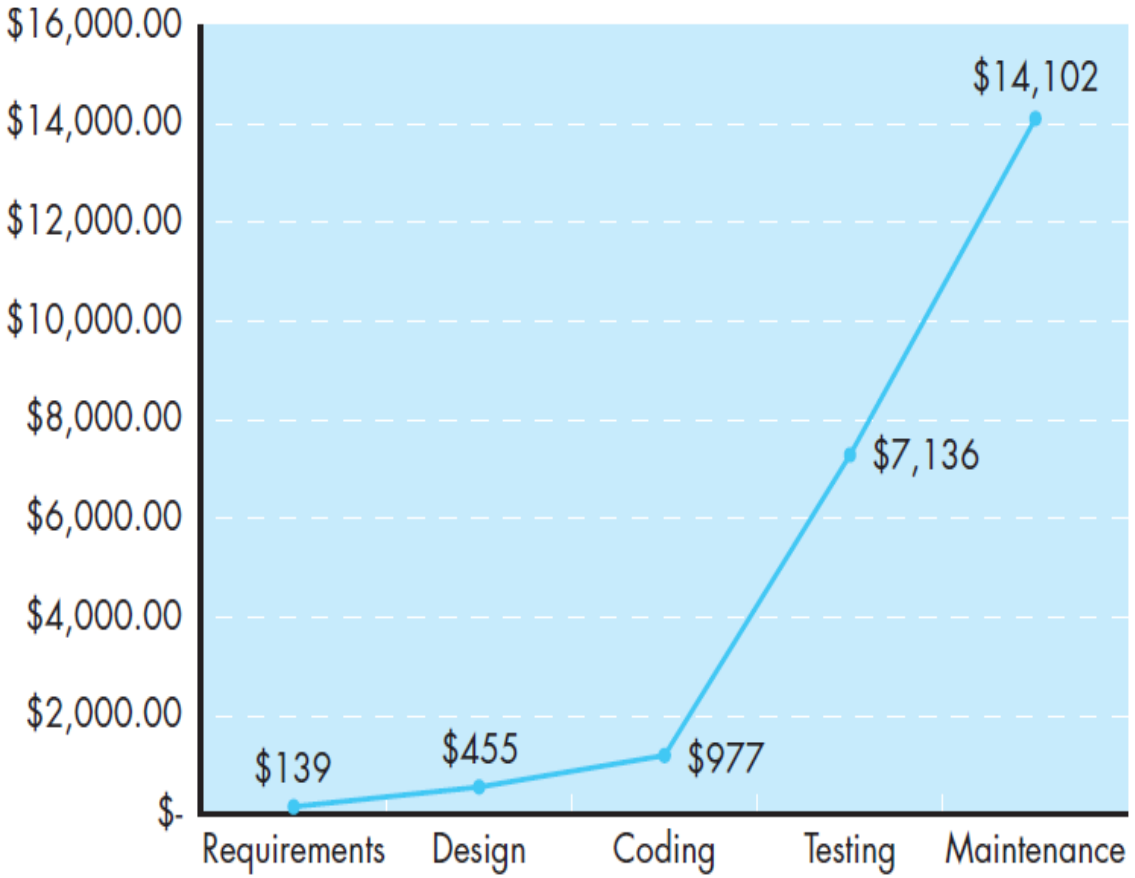
Class Discussion

- *Error*—vs.—*Defect*

- *We make this distinction because errors and defects have very different economic, business, psychological, and human impact*

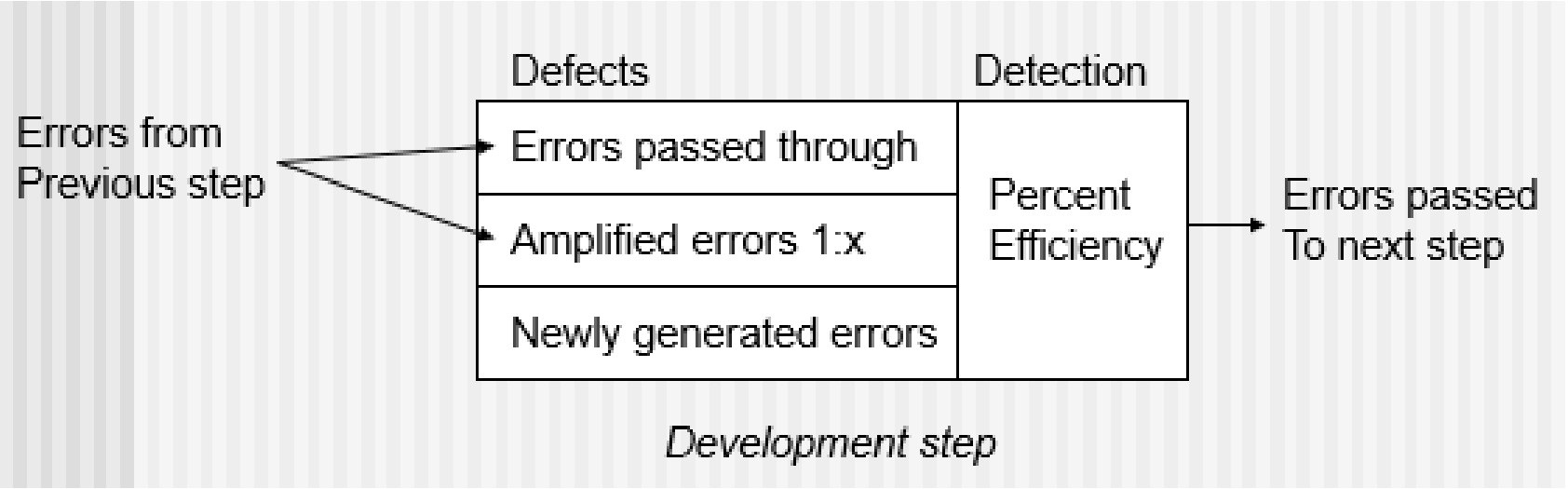
Relative cost of correcting errors and defects!

Relative cost of correcting errors and defects
Source: Adapted from [Boe01b].



Defect Amplification

- **A *defect amplification model*** [IBM81] can be used to illustrate the **generation** and **detection** of errors during the design and code generation actions of a software process.



Software Reliability Calculation

Scenario:

Assume that 10 errors have been introduced in the requirements model and that each error will be amplified by a factor of 2:1 into design and an addition 20 design errors are introduced and then amplified 1.5:1 into code where an additional 30 errors are introduced. Assume further that all unit testing will find 30 percent of all errors, integration will find 30 percent of the remaining errors, and validation tests will find 50 percent of the remaining errors. (No reviews are conducted).

Reconsider the situation described in the above scenario, but now assume that requirements, design, and code reviews are conducted and are 60 percent effective in uncovering all errors at that step.

Software Reliability Calculation

Solutions:

Calculate Total Errors After Amplification:

From Requirements to Design:

Amplified errors: 10 req. errors * 2 = 20 errors

Plus, new design errors: +20

Total Design Errors: 20 + 20 = 40 errors

From Design to Code:

Amplified errors: 40 design errors *1.5 = 60 errors

Plus, new code errors: +30

Total Code Errors (before testing): 60 + 30 = **90 errors**

Software Reliability Calculation

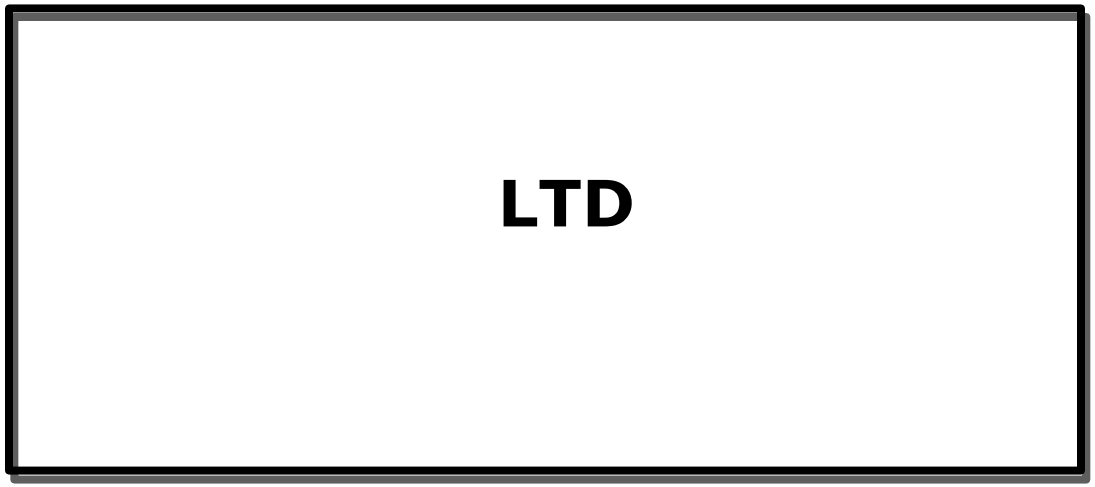
Testing:
Unit testing finds 30% of errors: $0.3 \times 90 = \mathbf{27}$ **errors** found. Remaining errors: $90 - 27 = \mathbf{63}$ **errors**.

Integration testing finds 30% of remaining errors: $0.3 \times 63 = \mathbf{18.9}$ **errors**.
Remaining errors: $63 - 18.9 = 44.1$.

Validation testing finds 50% of remaining errors: $0.5 \times 44.1 = \mathbf{22.05}$ **errors**.
Remaining errors: $44.1 - 22.05 = 22.05$.

Errors remaining after all testing: 22.05

— Tutorials —



Error in the process Software Reliability— **Tutorial**

- Same question but assume that requirements, design, and code reviews are conducted and are 34 percent effective in uncovering all errors at that step. How many errors will be released to the field?

Error in the process Software Reliability— **Tutorial**

- Suppose a software project initially contains 15 errors in its requirements model. Each error will propagate with a factor of 3:1 into the design phase, resulting in an additional 45 design errors. These design errors will then be magnified by a factor of 1.8:1 into the code, introducing an extra 81 errors. Assume that during unit testing, 40 percent of all errors are detected, followed by integration testing uncovering 25 percent of the remaining errors, and finally, validation tests finding 60 percent of the remaining errors. If no reviews are conducted, how many errors will persist and be released into the field?

Reliability Prediction Using Exponential Model — **Tutorial**

Scenario:

A medical monitoring device has a failure rate (λ) of 0.0002 failures/hour.

(a) Calculate the probability that the device operates without failure for 1000 hours.

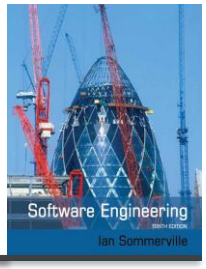
(c) If regulatory standards require 85% reliability at 1000 hours, does the device pass?

Further Reading!

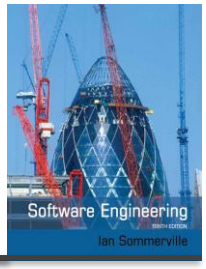
Prescribed Textbooks:

(a) Software Engineering, by Ian Sommerville 10th edition, Pearson Education Limited, 2016.

(b) Software Engineering – A Practitioner's Approach 8th edition Roger Pressman McGraw Hill Education (2015).

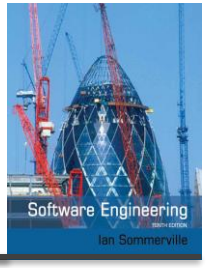


Chapter 8 – Software Testing



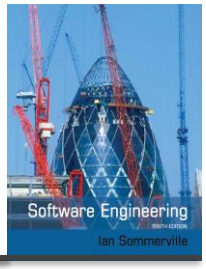
T3 Project!

Topics covered



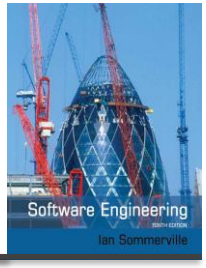
- ✧ Development testing
- ✧ Test-driven development
- ✧ Release testing
- ✧ User testing

Revision!



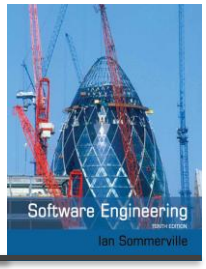
- What is the international standards that guide the organizations that design, develop and maintain products, including software?

Revision!



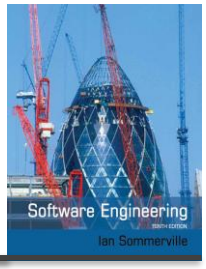
✧ What should Quality managers aim to develop?

Revision!



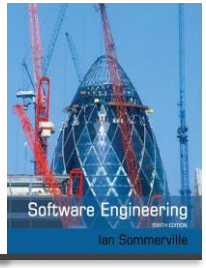
✧ What are the Software quality attributes?

Revision!



- Discuss this 'fitness for purpose' rather than specification conformance.

Catastrophic Effects of Poor Software Quality!



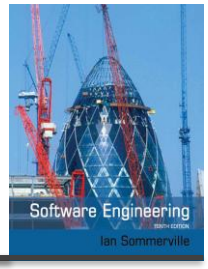
✧ Loss of Human Lives:

- **Therac-25 Radiation Machine (1985–87):** A software bug caused radiation overdoses, killing 3 patients and injuring others.
- **Boeing 737 MAX Crashes (2018–19):** Faulty flight control software (MCAS) contributed to two plane crashes, killing 346 people.

✧ Massive Financial Losses:

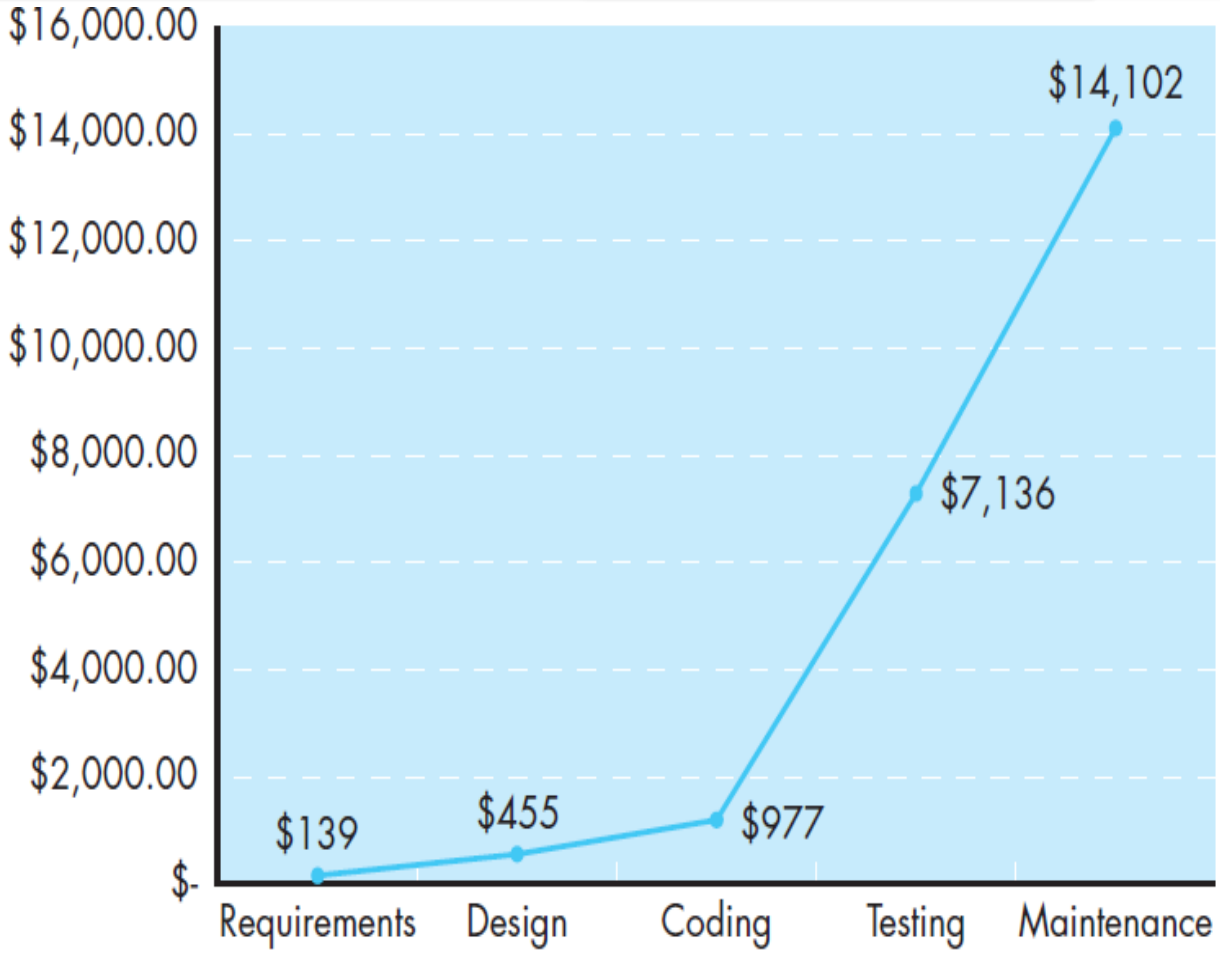
- **Knight Capital Group (2012):** A software glitch in a trading algorithm caused $440 million in losses in 45 minutes, nearly bankrupting the firm.
- **Ariane 5 Rocket Explosion (1996):** A software conversion error caused the European rocket to self-destruct shortly after launch, resulting in a $370 million loss.

Relative cost of correcting errors and defects!

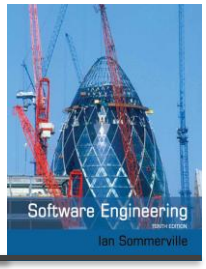


Relative cost of correcting errors and defects

Source: Adapted from [Boe01b].

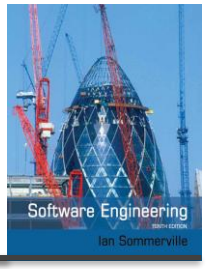


Program testing



- **Testing** is intended to show that
  - a program does what it is intended to do and
  - to discover program defects before it is put into use.
- **When you test software**,
  - you execute a program using artificial data.
- You check the results of the test run for errors, anomalies or information about the program's non-functional attributes.
- **Testing** is part of a more general verification and validation process
  - also includes static validation techniques.

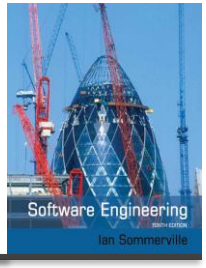
Program testing goals



- To demonstrate to the developer and the customer that the software meets its requirements.
  - **For custom software**, this means that there should be at least one test for every requirement in the requirements document.
  - **For generic software products**, it means that there should be tests for all the system features, plus combinations of these features, that will be incorporated in the product release.

  - **Defect testing** is concerned with rooting out undesirable system behavior. such as:
    - system crashes,
    - unwanted interactions with other systems,
    - incorrect computations and data corruption.

Testing process goals



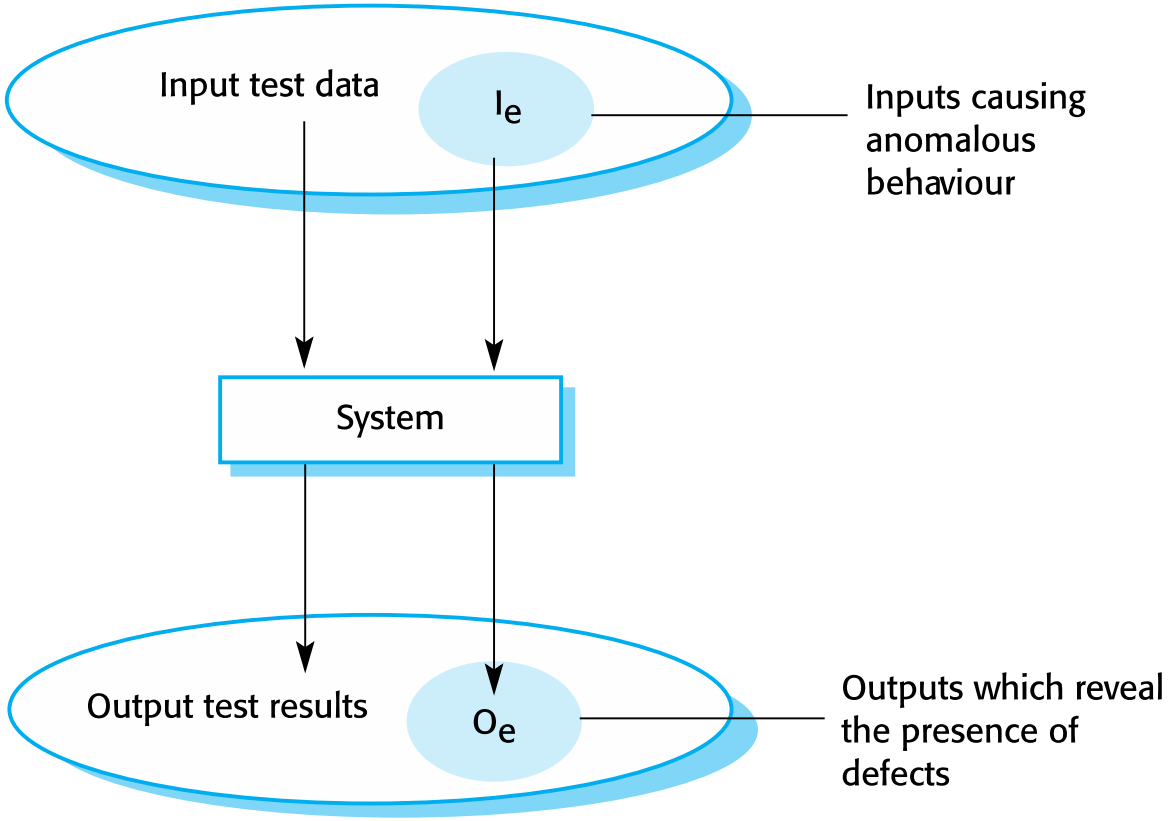
✧ Validation testing - First goal

- To demonstrate to the developer and the system customer that the software meets its requirements.
- You expect the system to perform correctly using a given set of test cases are designed to reflect the system's expected use.
- A successful test shows that the system operates as intended.

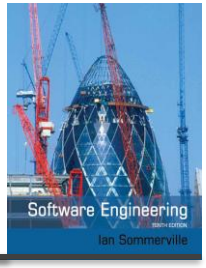
✧ Defect testing - Second goal

- To discover faults or defects in the software where its behaviour is incorrect or not in conformance with its specification
- The test cases are designed to expose defects.
- A successful test is a test that exposes a defect in the system.

An input-output model of program testing



Verification vs validation



✧ **Verification**:

"*Are we building the product right*".

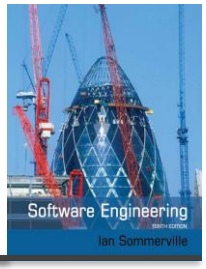
- The software should conform to its specification.

✧ **Validation**:

"*Are we building the right product*".

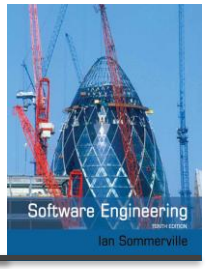
- The software should do what the user really requires.

V & V confidence



- Aim of **V & V** is to establish confidence that the system is 'fit for purpose'.
- Depends on system's purpose, user expectations and marketing environment
  - **Software purpose**
    - The level of confidence depends on how critical the software is to an organisation.
  - **User expectations**
    - Users may have low expectations of certain kinds of software.
  - **Marketing environment**
    - Getting a product to market early may be more important than finding defects in the program.

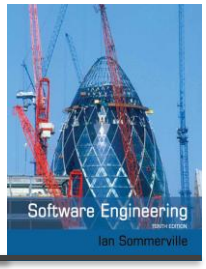
Inspections and Testing



- **Software inspections -** Concerned with analysis of the static system representation to discover problems
  - Static verification Technique - analyze code without running it

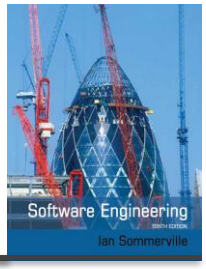
- **Software testing** - Concerned with exercising and observing product behaviour
  - Dynamic verification Technique - executing software under various conditions to identify errors, bugs, and performance issues.
  - Focuses on the behavior of the system during execution.

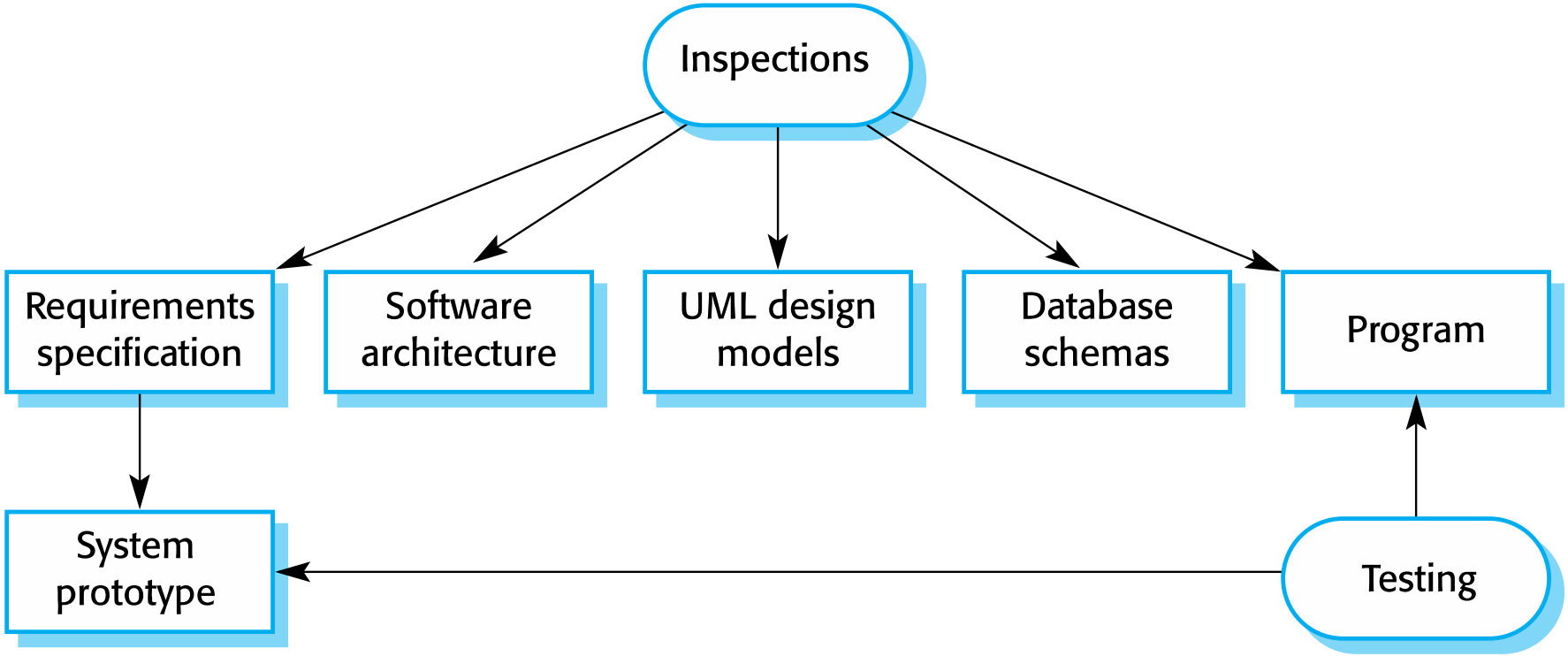
Software inspections



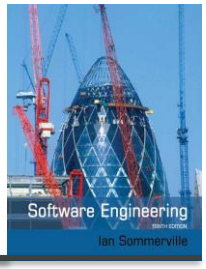
- These involve people examining the source representation with the aim of discovering anomalies and defects.
- They may be applied to any representation of the system (requirements, design, configuration data, test data, etc.).
- They have been shown to be an effective technique for discovering program errors.

Inspections and testing



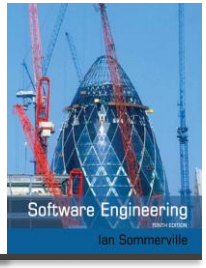


Advantages of inspections



- During testing, errors can mask (hide) other errors.
- Because inspection is a static process, you don't have to be concerned with interactions between errors.
- An inspection can be considered a broader quality attributes of a program
  - compliance with standards, portability and maintainability.

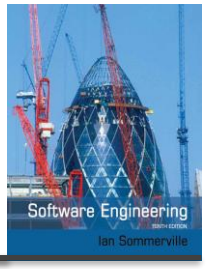
Inspections and testing



- Inspections and testing are complementary and not opposing verification techniques.

- Both should be used during the V & V process.

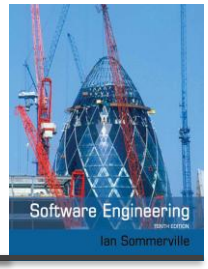
Inspection's Limitation

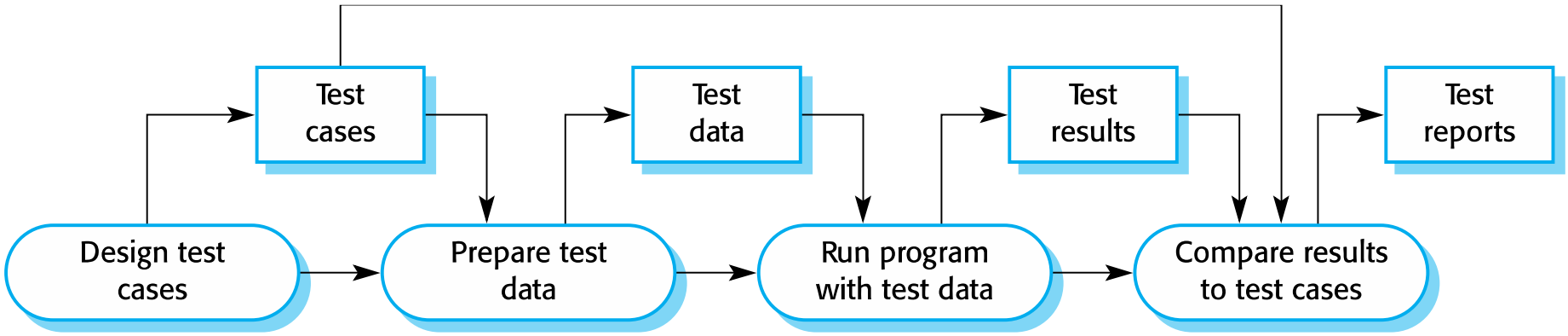


- Inspections can check conformance with a specification but not conformance with the customer's real requirements.

- Inspections cannot check non-functional characteristics
  - such as performance, usability, etc.

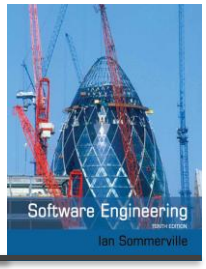
A model of the software testing process



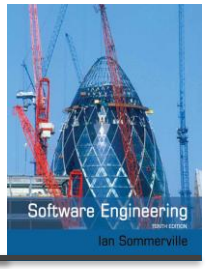


The **test cases** should show that, when used as expected, the component that you are testing does what it is supposed to do.

Stages of testing

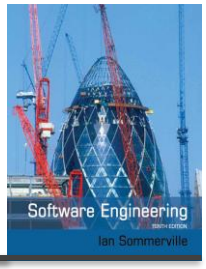


- **Development testing** - system is tested during development to discover bugs and defects.
- **Release testing** - a separate testing team test a complete version of the system before it is released to users.
- **User testing** - users or potential users of a system test the system in their own environment.



Development testing

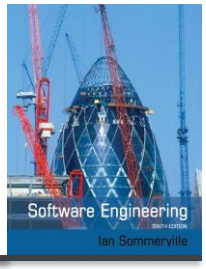
Development testing



Development testing includes all testing activities that are carried out by the team developing the system.

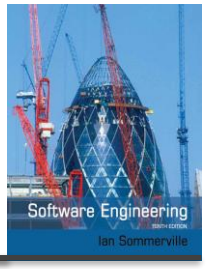
i. **System testing**
ii. **Component testing**
iii. **Unit testing**

System testing



- **System testing** - some or all of the components in a system are integrated and the system is tested as a whole.
  - The focus in system testing is testing the interactions between components.
  - System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.

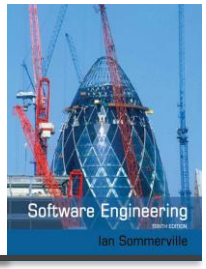
Component testing



- **Component testing** - several individual units are integrated to create composite components.

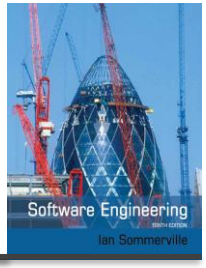
- The focus on component testing to test the interactions between units.

Unit testing



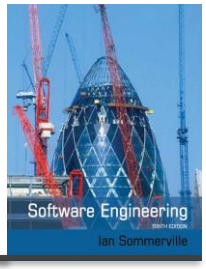
- **Unit testing -** process of testing individual components in isolation.
  - focus on testing the functionality of objects or methods.
    - Can be **automated -** tests are run and checked without manual intervention.

Detecting defects in the component



- **How do you detect defects in the component?**
- This leads to 2 types of unit test case:
  - The first of these should reflect normal operation of a program and should show that the component works as expected.
  - The other kind of test case should be based on testing experience of where common problems arise.
    - It should use abnormal inputs to check that these are properly processed and do not crash the component.

Testing strategies



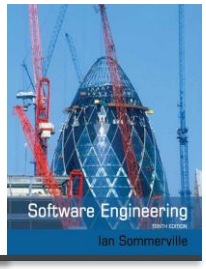
i. **Partition testing**

ii. **Guideline-based testing**

✧ **Partition testing** - identify groups of inputs that have common characteristics and should be processed in the same way.

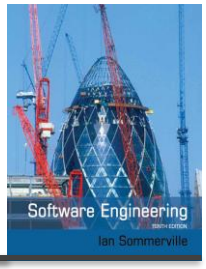
- Choose tests from within each of these groups.

Testing guidelines (sequences)

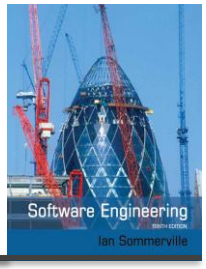


- **Guideline-based testing** - use testing guidelines to choose test cases.
  - These guidelines reflect previous experience of the kinds of errors that programmers often make when developing components.
  - Use sequences of different sizes in different tests.
  - Derive tests so that the first, middle and last elements of the sequence are accessed.
  - Test with sequences of zero length.

General testing guidelines

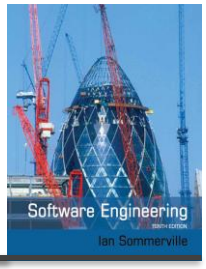


- Choose inputs that force the system to generate all error messages
- Design inputs that cause input buffers to overflow
- Repeat the same input or series of inputs numerous times
- Force invalid outputs to be generated
- Force computation results to be too large or too small.



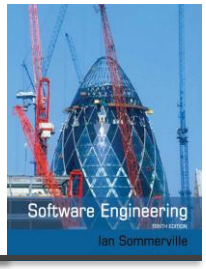
Test-driven development

Test-driven development



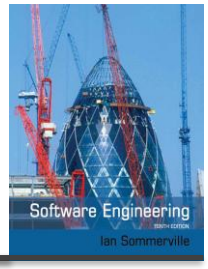
- **Test-driven development (TDD)** approach
  - inter-leave testing and code development.
- **Tests are written before code**
  - 'passing' the tests is the critical driver of development.
- **Develop code incrementally, along with a test**
  - You don't move on to the next increment until the code that you have developed passes its test.
- **TDD was introduced as part of agile methods**
  - such as Extreme Programming.
  - It can also be used in plan-driven development processes.

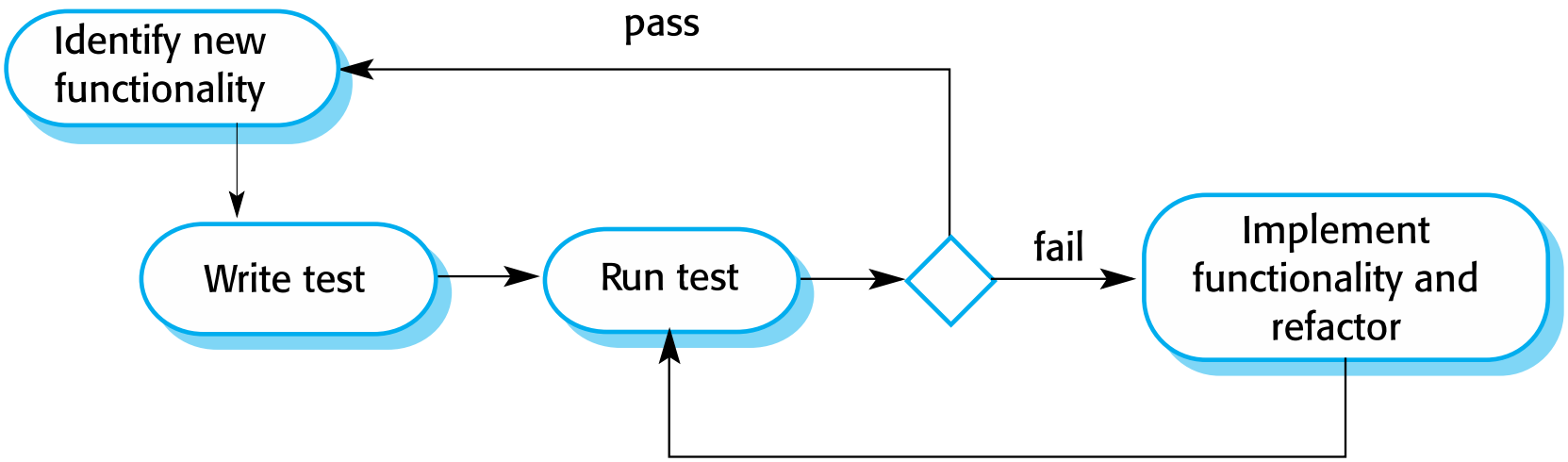
TDD process activities



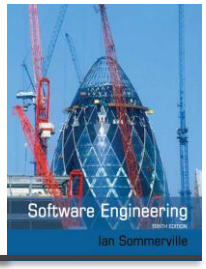
- Start by identifying the increment of functionality that is required.
  - Normally be small & implementable in a few lines of code.
- Write a test for this functionality and implement this as an automated test.
- Run the test, along with all other tests that have been implemented.
  - Initially, you have not implemented the functionality so the new test will fail.
- Implement the functionality and re-run the test.
- Once all tests run successfully, you move on to implementing the next chunk of functionality.

Test-driven development





Benefits of test-driven development



✧ Code coverage

- Every code segment that you write has at least one associated test, so all code written has at least one test.

✧ Regression testing

- All tests are rerun every time a change is made to the program
- ensures that recent code changes have not adversely affected existing functionalities

✧ Simplified debugging

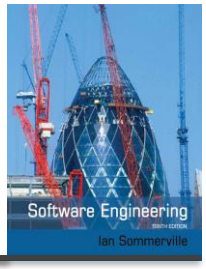
- When a test fails, it should be obvious where the problem lies. The newly written code needs to be checked and modified.

✧ System documentation

- The tests themselves are a form of documentation that describe what the code should be doing.

Release testing

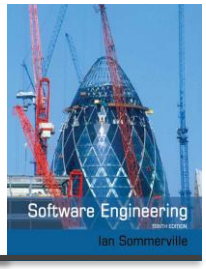
Release testing



- **Release testing** is the process of testing a particular release of a system that is intended for use outside of the development team.
- The **primary goal** of the release testing process - convince the supplier of the system that it is "**good enough**" for use.
  - Release testing, therefore, has to show that the system delivers its **specified functionality**, **performance** and **dependability**, and that it does **not fail during normal use**.
- Release testing is usually a **black-box testing process**
  - Tests are only derived from the system specification.

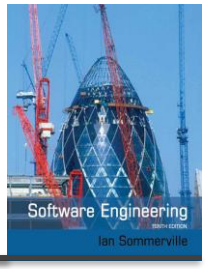
User testing

User testing



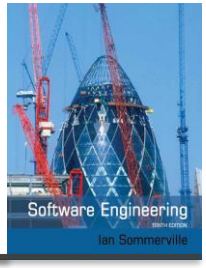
- User or customer testing is a stage in the testing process in which users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
  - The reason for this is that influences from the user's working environment have a **major effect** on:
  - the reliability,
  - performance,
  - usability and
  - robustness of a system.

Types of user testing



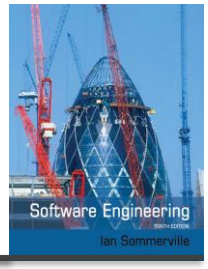
- **Alpha testing**
  - Users of the software work with the development team to test the software at the developer's site.
- **Beta testing**
  - A release of the software is made available to users to allow them to experiment and to raise problems that they discover with the system developers.
- **Acceptance testing**
  - Customers test a system to decide whether or not it is ready to be accepted.

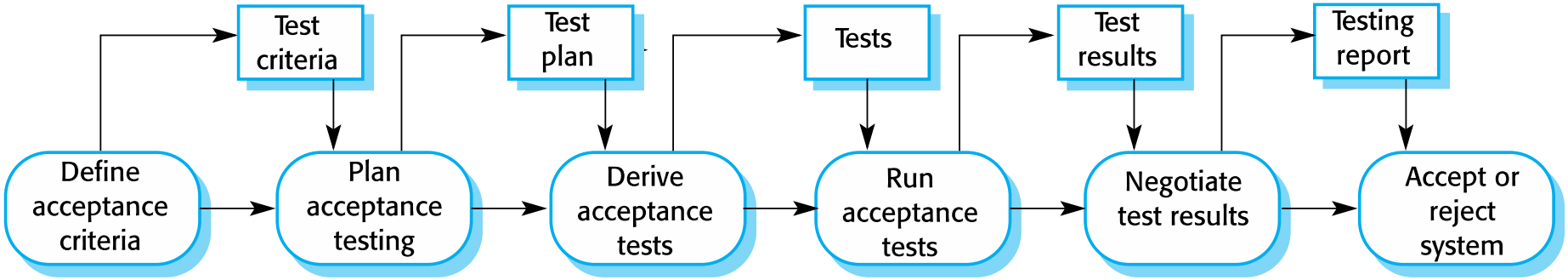
Stages in the acceptance testing process



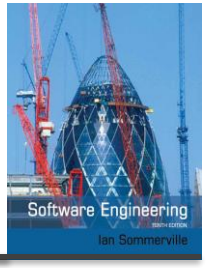
- ✧ Define acceptance criteria
- ✧ Plan acceptance testing
- ✧ Derive acceptance tests
- ✧ Run acceptance tests
- ✧ Negotiate test results
- ✧ Reject/accept system

The acceptance testing process



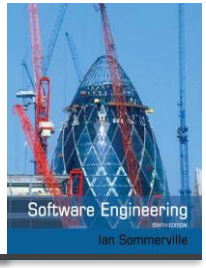


LTD-01!



- How does release testing differ from system testing?
- Discuss requirements-based testing and how it differs from performance testing.

LTD:

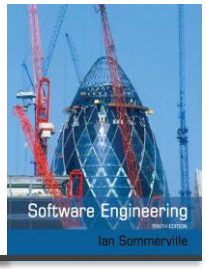


- ✧ Read about Agile methods and acceptance testing.

- ✧ Testing reveals defects but cannot prove their absence. How can software teams ensure reliability beyond traditional testing?

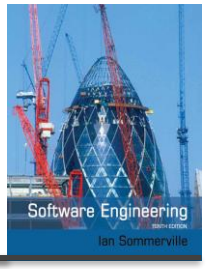
- ✧

Agile methods and acceptance testing



- In **agile methods**, the user/customer is part of the development team and is responsible for making decisions on the acceptability of the system.
- Tests are defined by the user/customer and are integrated with other tests in that they are run automatically when changes are made.
- There is no separate acceptance testing process.
- **Main problem** here is whether or not the embedded user is 'typical' and can represent the interests of all system stakeholders.

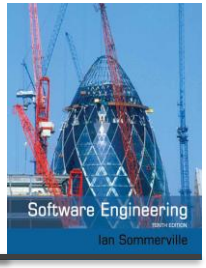
Key points



✧ Testing and Stakeholders

- **Development testing** is the responsibility of the **software development team / Developers.**
- Acceptance testing is **End user** testing process where the aim is to decide if the software is good enough to be deployed and used in its operational environment.
- A **separate team (QA Analysts/Testers)** should be responsible for testing a system before it is released to customers.
  - Design and execute manual & automated test cases. Perform functional, integration, system, and regression testing.
- **Performance Engineers (Performance & Load Testing)** –
  - Test scalability, speed, and stability over/under load.
- Project Managers / Scrum Masters –
  - Ensure testing is aligned with project timelines.

Key points

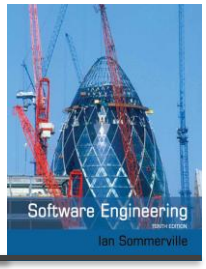


- When testing software, you should try to ‘break’ the software by using experience and guidelines
  - choose types of test case that have been effective in discovering defects
- Wherever possible, you should write automated tests.
  - embedded in a program that can be run every time a change is made to a system.

- **Limitations of Testing: Dijkstra’s principle ("Testing shows the presence, not the absence of bugs") impact quality assurance!**

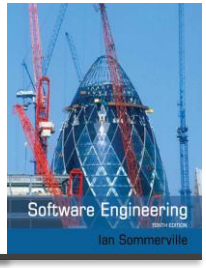
- **LTD: Testing reveals defects but cannot prove their absence. How can software teams ensure reliability beyond traditional testing?**

Scenario-Based Question – Software Testing Stakeholders



- You are the lead QA Analyst in a software development project nearing completion. The development team has completed their coding and conducted internal development testing. The Project Manager and Scrum Master are under pressure to meet a tight deadline and are requesting an early release.
- However:
  - Your QA team has not yet completed the planned system and regression testing.
  - End users have not performed acceptance testing.
  - Performance Engineers have raised concerns about occasional instability during load testing.

Question 1: Multiple Choice



- Which of the following stakeholder groups is primarily responsible for validating the software from a user and business perspective before release?

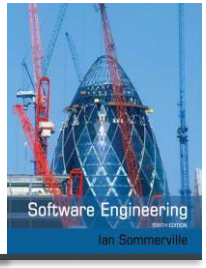
A. Developers

B. QA Analysts

C. End Users

D. Performance Engineers

Question 2: True or False

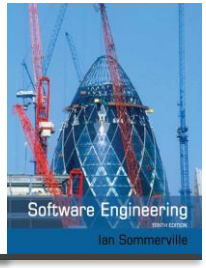


- It is acceptable to skip regression testing if development testing has passed and the release date is near.

A. True

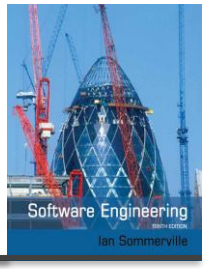
B. False

LTD: Question 3: Scenario Discussion



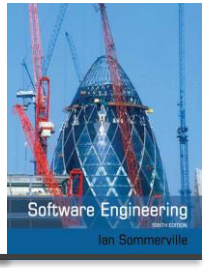
i. What actions would you take to balance the need for quality assurance with the project timeline pressure?

ii. List at least three key steps and explain which stakeholders you would engage as their roles.



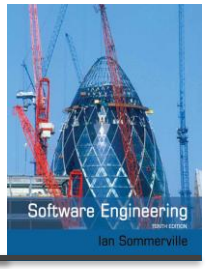
Chapter 11 – Reliability Engineering

Topics covered



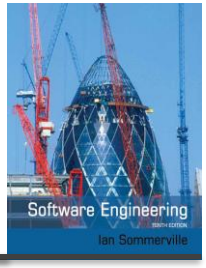
- ✧ Availability and reliability
- ✧ Reliability requirements
- ✧ Fault-tolerant architectures
- ✧ Programming for reliability
- ✧ Reliability measurement

Software reliability



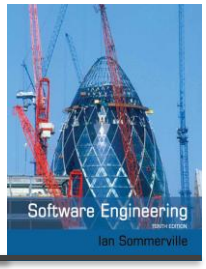
- In general, software customers expect all software to be dependable.
  - However, for **non-critical applications** - willing to accept some system failures.
- **critical systems -** have very high reliability requirements and special software engineering techniques may be used to achieve this.
  - Medical systems
  - Telecommunications and power systems
  - Aerospace systems

Faults, errors and failures



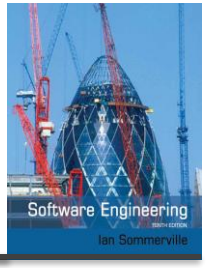
Term	Description
Human error or mistake	Human behavior that results in the introduction of faults into a system. For example, in the wilderness weather system, a programmer might decide that the way to compute the time for the next transmission is to add 1 hour to the current time. This works except when the transmission time is between 23.00 and midnight (midnight is 00.00 in the 24-hour clock).
System fault	A characteristic of a software system that can lead to a system error. The fault is the inclusion of the code to add 1 hour to the time of the last transmission, without a check if the time is greater than or equal to 23.00.
System error	An erroneous system state that can lead to system behavior that is unexpected by system users. The value of transmission time is set incorrectly (to 24.XX rather than 00.XX) when the faulty code is executed.
System failure	An event that occurs at some point in time when the system does not deliver a service as expected by its users. No weather data is transmitted because the time is invalid.

Faults and failures



- **Failures** are a usually a result of system errors that are derived from faults in the system
- However, **faults do not necessarily result in system errors**
  - The erroneous system state resulting from the fault may be transient and 'corrected' before an error arises.
  - The faulty code may never be executed.
- **Errors do not necessarily lead to system failures**
  - The error can be corrected by built-in error detection and recovery
  - The failure can be protected against by built-in protection facilities. These may, for example, protect system resources from system errors

Fault management



✧ Fault avoidance

- The system is developed in such a way that human error is avoided and thus system faults are minimised.
- The development process is organised so that faults in the system are detected and repaired before delivery to the customer.

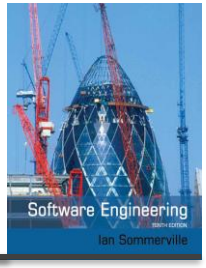
✧ Fault detection

- Verification and validation techniques are used to discover and remove faults in a system before it is deployed.

✧ Fault tolerance

- The system is designed so that faults in the delivered software do not result in system failure.

Reliability achievement



✧ Fault avoidance

- Development technique are used that either minimise the possibility of mistakes or trap mistakes before they result in the introduction of system faults.

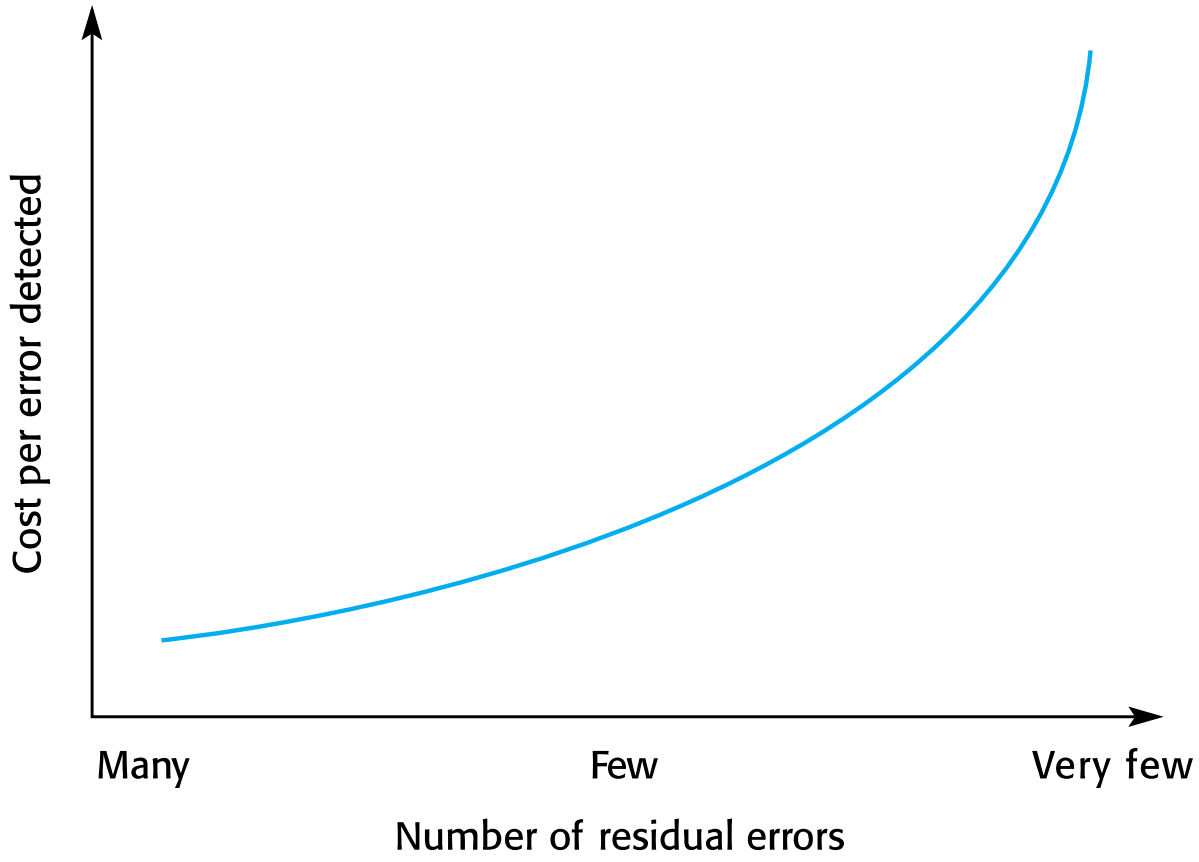
✧ Fault detection and removal

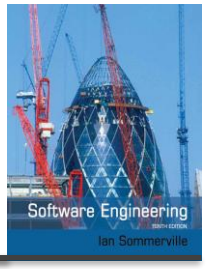
- Verification and validation techniques are used that increase the probability of detecting and correcting errors before the system goes into service are used.

✧ Fault tolerance

- Run-time techniques are used to ensure that system faults do not result in system errors and/or that system errors do not lead to system failures.

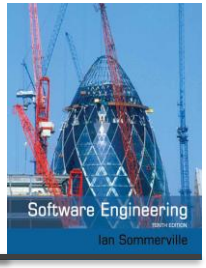
The increasing costs of residual fault removal





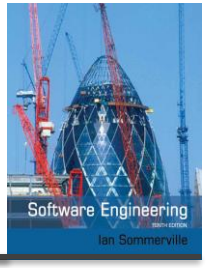
Availability and reliability

Availability and reliability



- **Reliability**
  - The probability of failure-free system operation over a specified time in a given environment for a given purpose
- **Availability**
  - The probability that a system, at a point in time, will be operational and able to deliver the requested services
- Both of these attributes can be expressed quantitatively e.g. availability of 0.999 means that the system is up and running for 99.9% of the time.

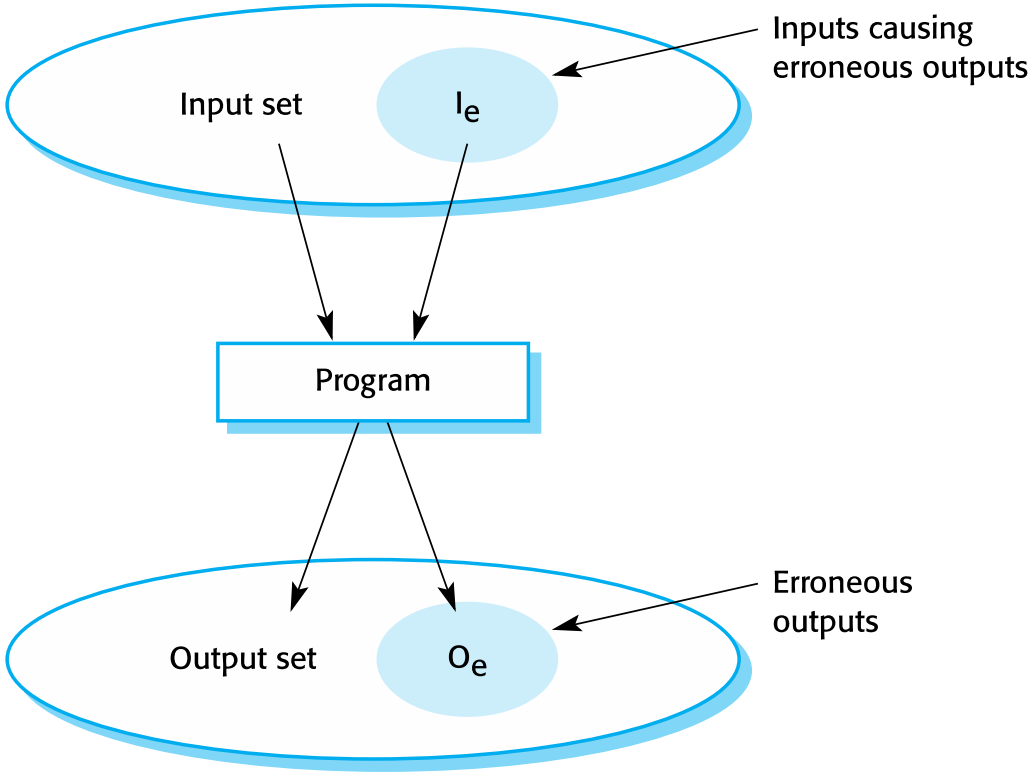
Reliability and specifications



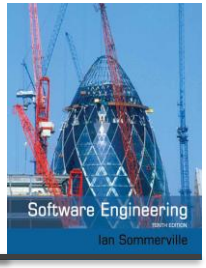
- **Reliability** can only be defined formally with respect to a system specification i.e. a failure is a deviation from a specification.

- Note that removing X% of the faults in a system will not necessarily improve the reliability by X%.

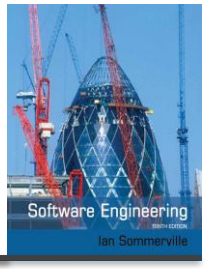
A system as an input/output mapping



Availability perception

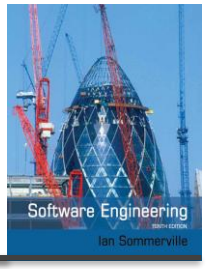


- Availability is usually expressed as a percentage of the time that the system is available to deliver services e.g. 99.95%.
- However, this does not take into account two factors:
  - **The number of users affected by the service outage**. Loss of service in the middle of the night is less important for many systems than loss of service during peak usage periods.
  - **The length of the outage**. The longer the outage, the more the disruption. Several short outages are less likely to be disruptive than 1 long outage. Long repair times are a particular problem.



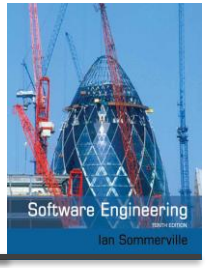
Reliability requirements

Reliability metrics



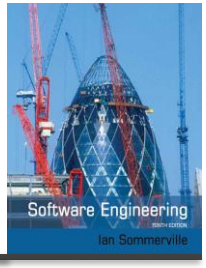
- Reliability metrics are units of measurement of system reliability.
- System reliability is measured by counting the number of operational failures and, where appropriate, relating these to the demands made on the system and the time that the system has been operational.
- A long-term measurement programme is required to assess the reliability of critical systems.
- **Metrics**
  - Probability of failure on demand
  - Rate of occurrence of failures/Mean time to failure
  - Availability

Probability of failure on demand (POFOD)



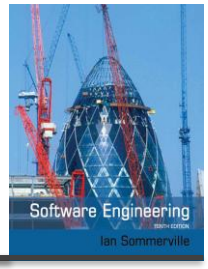
- This is the **probability that the system will fail when a service request is made**. Useful when demands for service are intermittent and relatively infrequent.
- Appropriate for protection systems where services are demanded occasionally and where there are serious consequence if the service is not delivered.
- Relevant for many safety-critical systems with exception management components
  - Emergency shutdown system in a chemical plant.

Availability



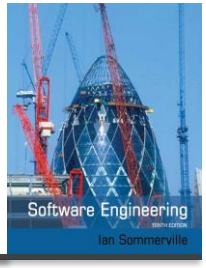
- **Measure of the fraction of the time that the system is available for use.**
- Takes repair and restart time into account
- Availability of **0.998 means software is available for 998 out of 1000 time units**.
- Relevant for non-stop, continuously running systems
  - telephone switching systems, railway signalling systems.

Availability specification



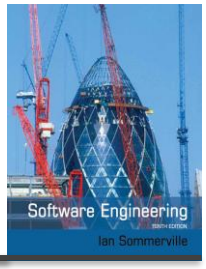
Availability	Explanation
0.9	The system is available for 90% of the time. This means that, in a 24-hour period (1,440 minutes), the system will be unavailable for 144 minutes.
0.99	In a 24-hour period, the system is unavailable for 14.4 minutes.
0.999	The system is unavailable for 84 seconds in a 24-hour period.
0.9999	The system is unavailable for 8.4 seconds in a 24-hour period. Roughly, one minute per week.

Non-functional reliability requirements



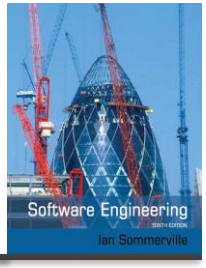
- Non-functional reliability requirements are specifications of the required reliability and availability of a system using one of the reliability metrics (POFOD, ROCOF or AVAIL).
- Quantitative reliability and availability specification has been used for many years in safety-critical systems but is uncommon for business critical systems.
- However, as more and more companies demand 24/7 service from their systems, it makes sense for them to be precise about their reliability and availability expectations.

Benefits of reliability specification



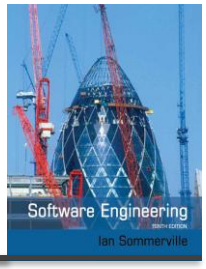
- The process of deciding the required level of the reliability helps to clarify what stakeholders really need.
- It provides a basis for assessing when to stop testing a system. You stop when the system has reached its required reliability level.
- It is a means of assessing different design strategies intended to improve the reliability of a system.
- If a regulator has to approve a system (e.g. all systems that are critical to flight safety on an aircraft are regulated), then evidence that a required reliability target has been met is important for system certification.

Specifying reliability requirements



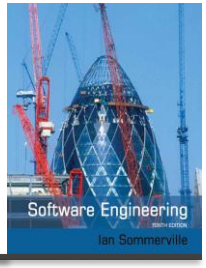
- Specify the availability and reliability requirements for different types of failure. There should be a lower probability of high-cost failures than failures that don't have serious consequences.
- Specify the availability and reliability requirements for different types of system service. Critical system services should have the highest reliability but you may be willing to tolerate more failures in less critical services.
- Think about whether a high level of reliability is really required. Other mechanisms can be used to provide reliable system service.

ATM reliability specification



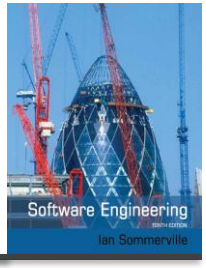
- Key concerns
  - To ensure that their ATMs carry out customer services as requested and that they properly record customer transactions in the account database.
  - To ensure that these ATM systems are available for use when required.
- Database transaction mechanisms may be used to correct transaction problems so a low-level of ATM reliability is all that is required
- Availability, in this case, is more important than reliability

ATM availability specification



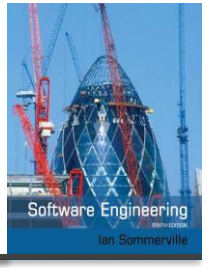
- System services
  - The customer account database service;
  - The individual services provided by an ATM such as ‘withdraw cash’, ‘provide account information’, etc.
- The database service is critical as failure of this service means that all of the ATMs in the network are out of action.
- You should specify this to have a high level of availability.
  - Database availability should be around 0.9999, between 7 am and 11pm.
  - This corresponds to a downtime of less than 1 minute per week.

ATM availability specification



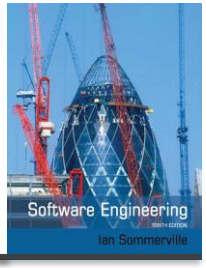
- For an individual ATM, the key reliability issues depends on mechanical reliability and the fact that it can run out of cash.
- A lower level of software availability for the ATM software is acceptable.
- The overall availability of the ATM software might therefore be specified as 0.999, which means that a machine might be unavailable for between 1 and 2 minutes each day.

Insulin pump reliability specification



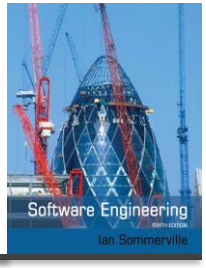
- Probability of failure (POFOD) is the most appropriate metric.
- Transient failures that can be repaired by user actions such as recalibration of the machine. A relatively low value of POFOD is acceptable (say 0.002) – one failure may occur in every 500 demands.
- Permanent failures require the software to be re-installed by the manufacturer. This should occur no more than once per year. POFOD for this situation should be less than 0.00002.

Functional reliability requirements



- Checking requirements that identify checks to ensure that incorrect data is detected before it leads to a failure.
- Recovery requirements that are geared to help the system recover after a failure has occurred.
- Redundancy requirements that specify redundant features of the system to be included.
- Process requirements for reliability which specify the development process to be used may also be included.

Examples of functional reliability requirements

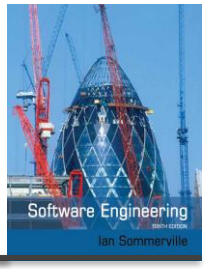


RR1: A pre-defined range for all operator inputs shall be defined and the system shall check that all operator inputs fall within this pre-defined range. (Checking)

RR2: Copies of the patient database shall be maintained on two separate servers that are not housed in the same building. (Recovery, redundancy)

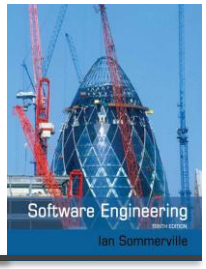
RR3: N-version programming shall be used to implement the braking control system. (Redundancy)

RR4: The system must be implemented in a safe subset of Ada and checked using static analysis. (Process)



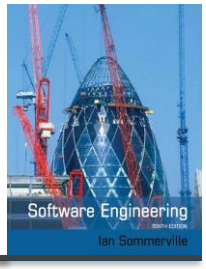
Fault-tolerant architectures

Fault tolerance



- **Fault tolerance**:
  - system can continue in operation in spite of software failure.
  - Even if the system has been proved to conform to its specification, it must also be fault tolerant as
    - there may be specification errors
    - the validation may be incorrect.

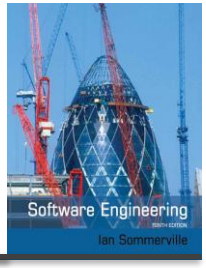
Fault tolerance



- **Fault tolerance is required:**
  - where there are **high availability requirements**
  - where **system failure costs are very high**.

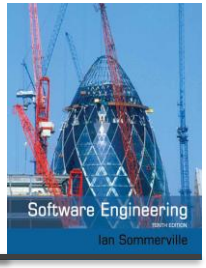
- In **critical situations**, software systems must be fault tolerant.

Fault-tolerant system architectures



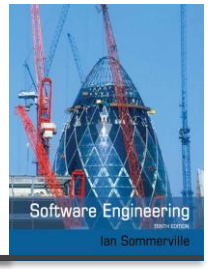
- **Fault-tolerant architectures**:
  - Generally, all based on **redundancy** and **diversity**.
- Examples of situations where dependable architectures are used:
  - **Flight control systems**, where system failure could threaten the safety of passengers
  - **Reactor systems** where failure of a control system could lead to a chemical or nuclear emergency
  - **Telecommunication systems**, where there is a need for 24/7 availability.

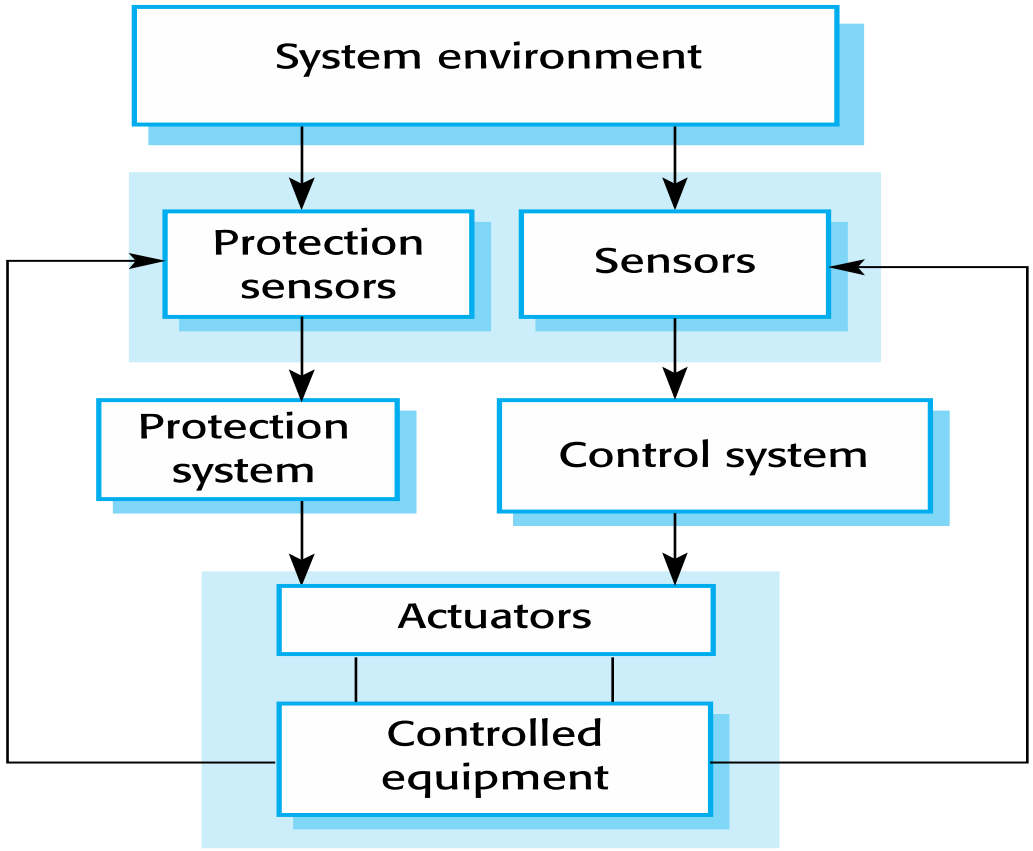
Protection systems



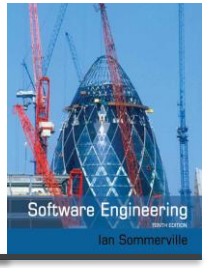
- **Protection systems -** independently monitor the controlled system and the environment.
- **If a problem is detected, it issues commands to**
  - take emergency action
  - shut down the system and avoid a catastrophe.
- **Example**
  - System to stop a train if it passes a red light
  - System to shut down a reactor if temperature/pressure are too high

Protection system architecture





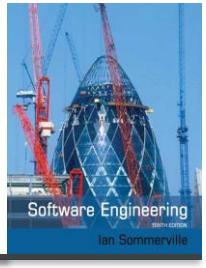
Protection system functionality



- **Protection systems**
  - **Redundant** - include monitoring and control capabilities.
  - **Diverse** and use different technology from the control software.

- **Aim** is to ensure that there is a low probability of failure on demand for the protection system.

Self-monitoring architectures



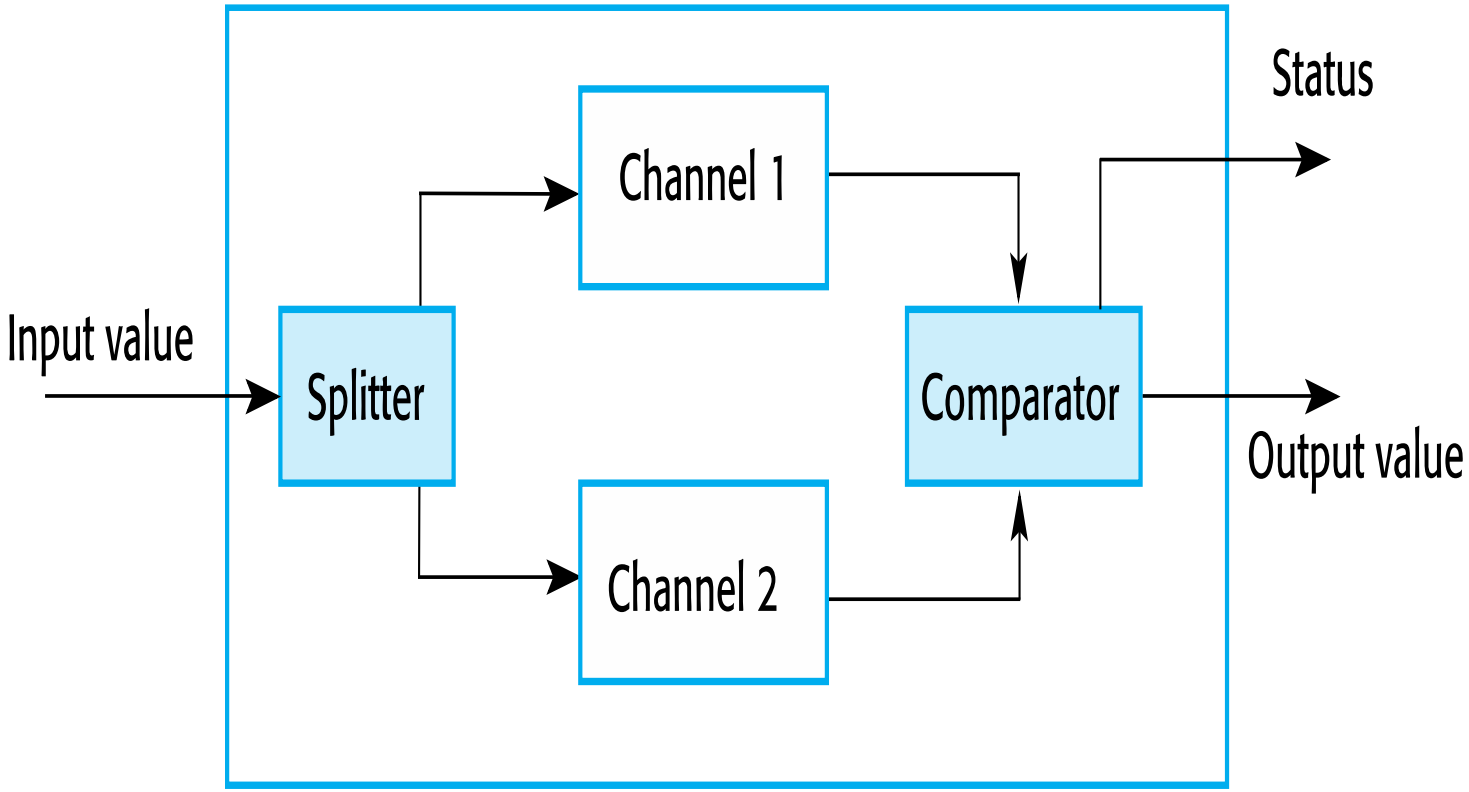
✧ Self-monitoring architectures:

- **Multi-channel architectures** where the system monitors its own operations and takes action

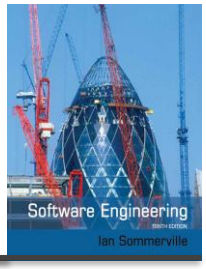
✧ How it works:

- The same computation is carried out on each channel and the results are compared.
- If the results are identical and are produced at the same time, then it is assumed that the system is operating correctly.
- If the results are different, then a failure is assumed, and a **failure exception** is raised.

Self-monitoring architecture



Self-monitoring systems



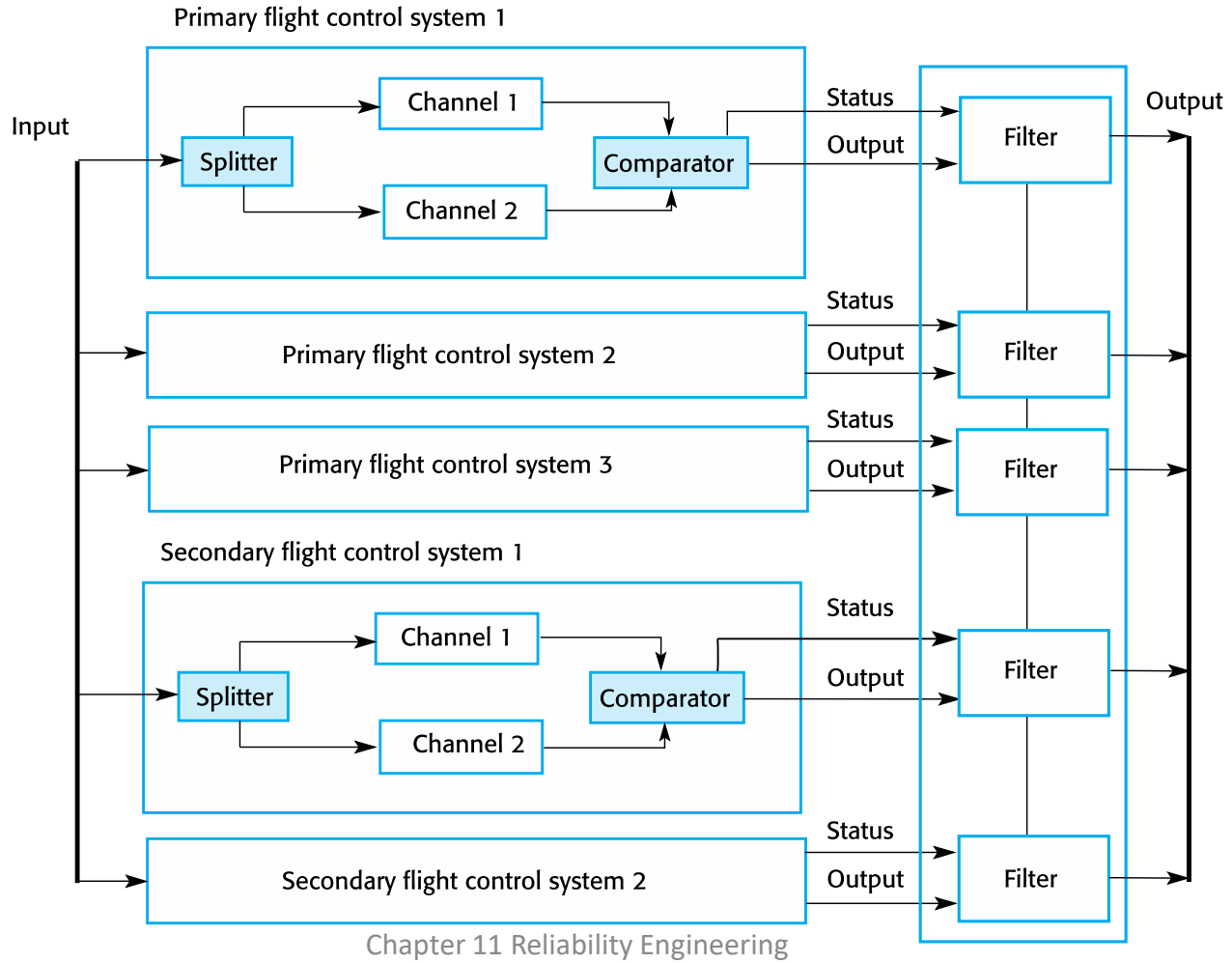
✧ If **high-availability** is required:

- use several self-checking systems in parallel.
- Approach is used in the Airbus family of aircraft for their flight control systems.

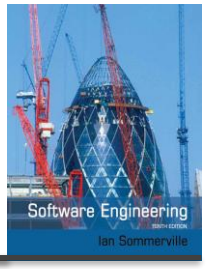
✧ Airbus flight control system architecture (Study)

Airbus flight control system architecture

Input value

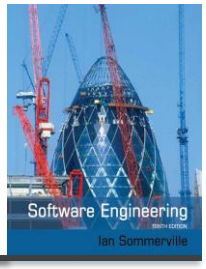


Airbus architecture discussion



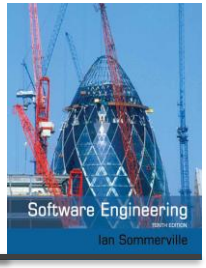
- The Airbus FCS has **5 separate computers**, any one of which can run the control software.
- Extensive use has been made of **diversity**
  - **Primary systems** use a **different processor** from the secondary systems.
  - **Primary and secondary systems** use **chipsets from different** manufacturers.
  - **Software in secondary systems** is **less complex** than in primary system – provides only critical functionality.
  - **Software in each channel** is developed in **different programming languages by different teams.**
  - **Different programming languages used** in **primary and secondary systems.**

N-version programming



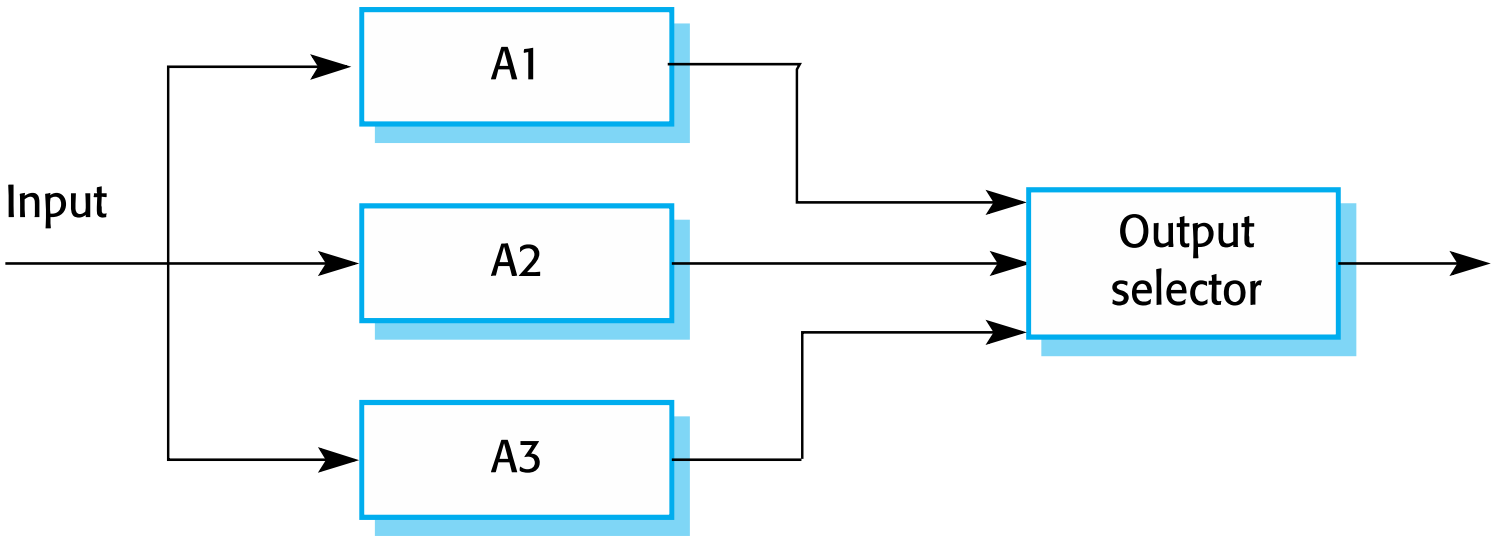
- **N-version programming:**
  - The **different system versions are designed and implemented by different teams**.
  - It is assumed that there is **a low probability that they will make the same mistakes**.
  - There is some **empirical evidence that teams commonly misinterpret specifications in the same way** and chose the same algorithms in their systems.

N-version programming

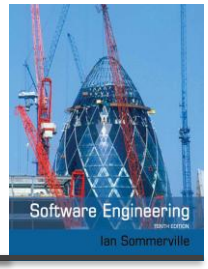


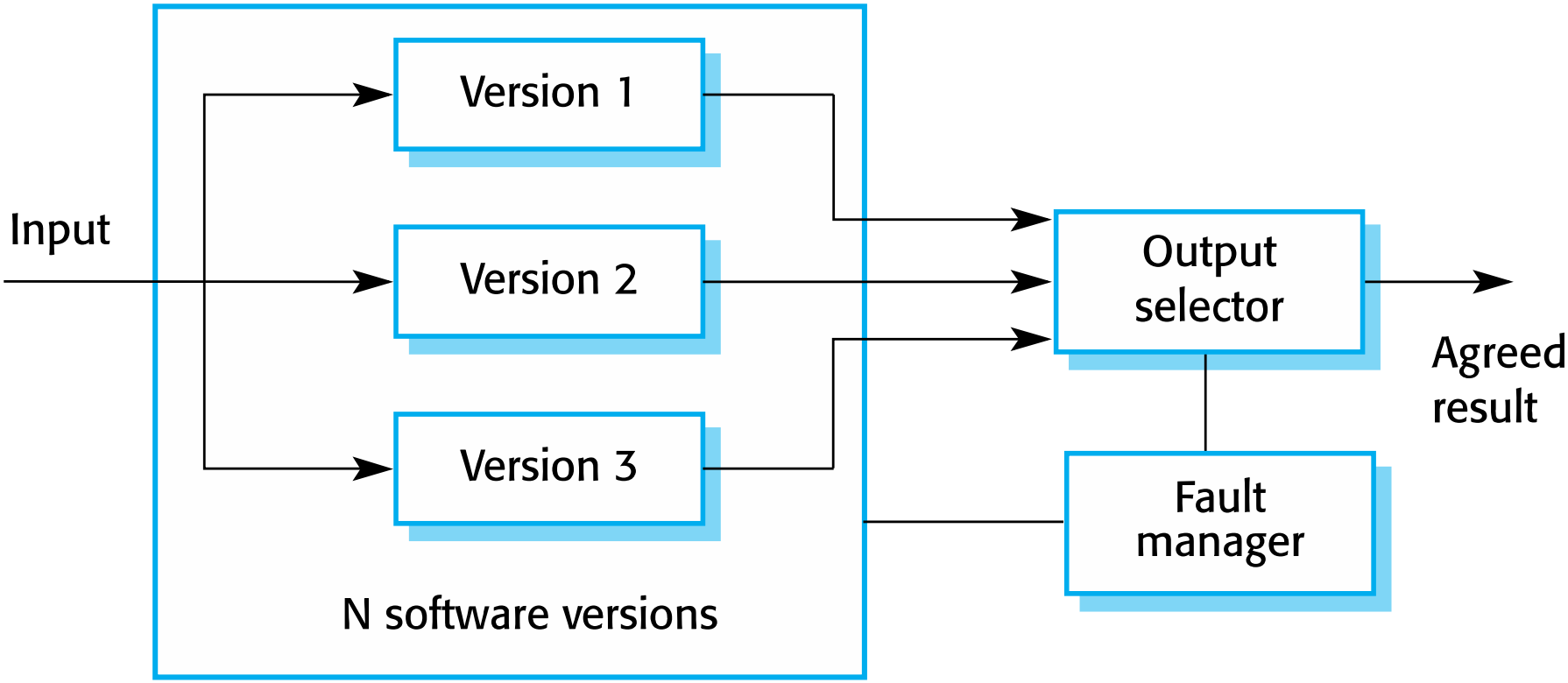
- **Multiple versions of a software system**
  - carry out **computations at the same time** - odd number of computers involved, typically 3.
  - The **results are compared using a voting system** - majority result is taken to be the correct result.
- **Triple-modular redundancy (TMR)** - Approach derived from the notion.

Triple modular redundancy

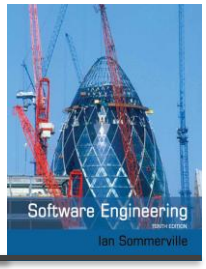


N-version programming





Software diversity



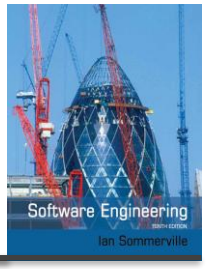
✧ Approaches to software fault tolerance

- **depend on software diversity** where it is assumed that different implementations of the same software specification will fail in different ways.
- It is assumed that implementations are
- (a) independent
- (b) do not include common errors.

✧ Strategies to achieve diversity

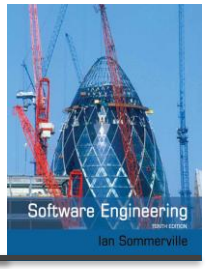
- Different programming languages
- Different design methods and tools
- Explicit specification of different algorithms

Problems with design diversity



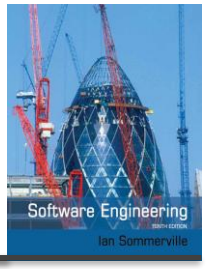
- Teams are not culturally diverse, so they tend to tackle problems in the same way.

- **Characteristic errors**
  - Different teams make the same mistakes.
  - Specification errors - if there is an error in the specification then this is reflected in all implementations



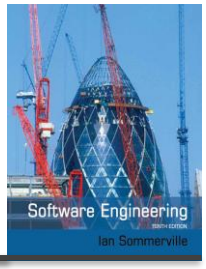
Programming for reliability

Dependable programming



- **To reduce the incidence of program faults.**
  - Good programming practices
- **The Programming practices support**
  - Fault avoidance
  - Fault detection
  - Fault tolerance

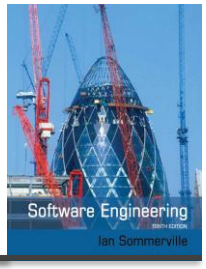
Good practice guidelines for dependable programming



Dependable programming guidelines

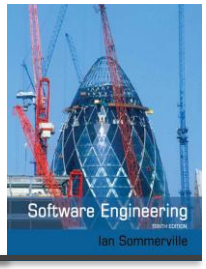
1. Limit the visibility of information in a program
2. Check all inputs for validity
3. Provide a handler for all exceptions
4. Minimize the use of error-prone constructs
5. Provide restart capabilities
6. Check array bounds
7. Include timeouts when calling external components
8. Name all constants that represent real-world values

(1) Limit the visibility of information in a program



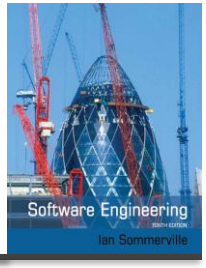
- **Program components** should only be allowed access to data that they need for their implementation.
  - This means that accidental corruption of parts of the program state by these components is impossible.

(2) Check all inputs for validity



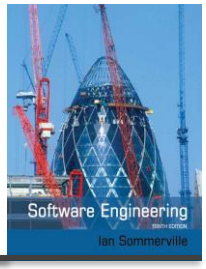
- **All program take inputs from their environment and make assumptions** about these inputs.
  - This will help inputs to be consistent with these assumptions.
  - Always check inputs before processing against the assumptions made about these inputs.

Validity checks



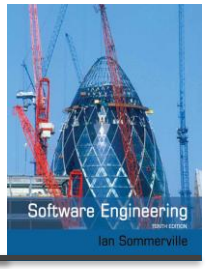
- **Range checks**
  - Check that the **input falls within a known range**.
- **Size checks**
  - Check that the **input does not exceed some maximum size** e.g. 40 characters for a name.
- **Representation checks**
  - Check that the **input does not include characters that should not be part of its representation** e.g. names do not include numerals.
- **Reasonable checks**
  - Use information about the **input to check if it is reasonable** rather than an extreme value.

(3) Provide a handler for all exceptions



- **A program exception**
  - an error or some unexpected event such as a **power failure**.
- **Exception handling constructs**
  - a mechanism to provide some fault tolerance – to detect exceptions.
- **Using normal control constructs**
  - to detect exceptions needs many additional statements to be added to the program.

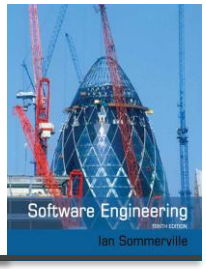
(4) Minimize the use of error-prone constructs



- **Program faults** are usually a consequence of human error
  - because programmers lose track of the relationships between the different parts of the system

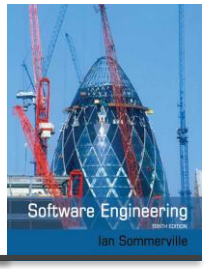
- Avoid or minimize the use of error-prone constructs!

Error-prone constructs



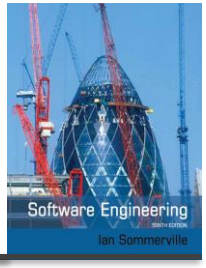
- **Unconditional branch (goto) statements**
- **Floating-point numbers**
  - The imprecision may lead to invalid comparisons.
- **Pointers referring to the wrong memory areas**
- **Dynamic memory allocation**
  - Run-time allocation can cause memory overflow.

Error-prone constructs



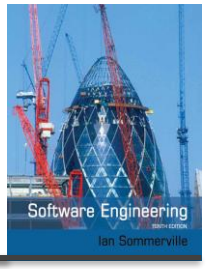
- **Parallelism**
  - Can result in subtle timing errors because of unforeseen interaction between parallel processes.
- **Recursion**
  - Errors in recursion can cause memory overflow as the program stack fills up.
- **Interrupts**
  - Interrupts can cause a critical operation to be terminated and make a program difficult to understand.
- **Inheritance**
  - Code is not localised. This can result in unexpected behaviour when changes are made

(5) Provide restart capabilities



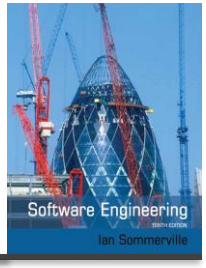
- **For systems that involve long transactions or user interactions**:
  - provide a restart capability that allows the system to restart after failure without users having to redo everything that they have done.
- **Restart depends on the type of system**
  - Keep copies of forms so that users don't have to fill them in again if there is a problem
  - Save state periodically and restart from the saved state

(6) Check array bounds



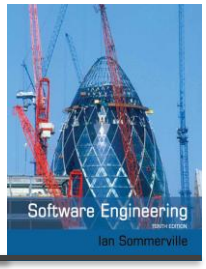
- **Always check that an array access is within the bounds of the array:**
  - Programming languages, such as C - it is possible to address a memory location outside of the range allowed for in an array declaration.
  - This may lead to the well-known '**bounded buffer**' **vulnerability**
    - where attackers may deliberately write executable code into memory.

(7) Include timeouts when calling external components

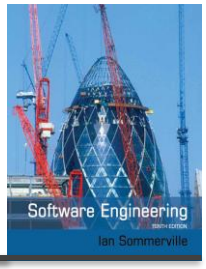


- **Always include timeouts on all calls to external components**.
  - After a defined time period has elapsed without a response, your system should then assume failure and take whatever actions are required to recover from this.

(8) Name all constants that represent real-world values

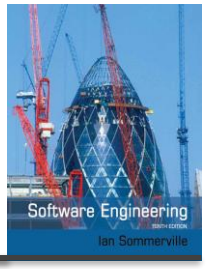


- Always give constants that reflect real-world values (such as tax rates) names rather than using their numeric values
  - always refer to them by name
  - when these ‘constants’ change (for sure, they are not really constant), then you only have to make the change in one place in your program.



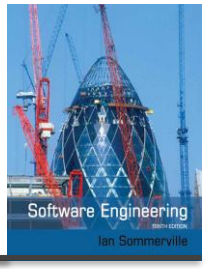
Reliability measurement

Reliability measurement



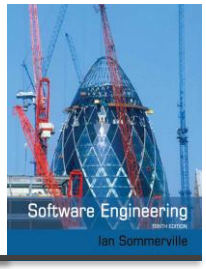
- **Reliability metrics** include
  - probability of failure on demand (POFOD),
  - rate of occurrence of failure (ROCOF) and
  - availability (AVAIL).
- **To assess the reliability of a system**, collect data about its operation. The data required may include:
  - The number of system failures given a number of requests for system services.
  - This is used to measure the **POFOD**. This applies irrespective of the time over which the demands are made.

Reliability measurement



- The time or the number of transactions between system failures plus the total elapsed time or total number of transactions.
  - This is used to measure **ROCOF** and **MTTF**(Mean Time To Failure)
- **MTTF** - reliability metric that measures the average time expected until the first failure of a piece of equipment or system under normal operating conditions
- This is used in the **measurement of availability**.
  - **Availability** does not just depend on the time between failures but also on the time required to get the system back into operation.

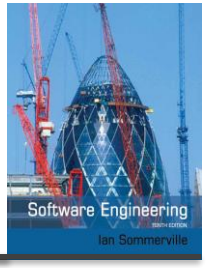
Reliability testing



✧ Reliability testing (Statistical testing)

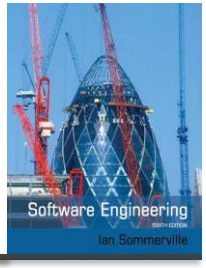
- involves running the program to assess whether or not it has reached the required level of reliability.

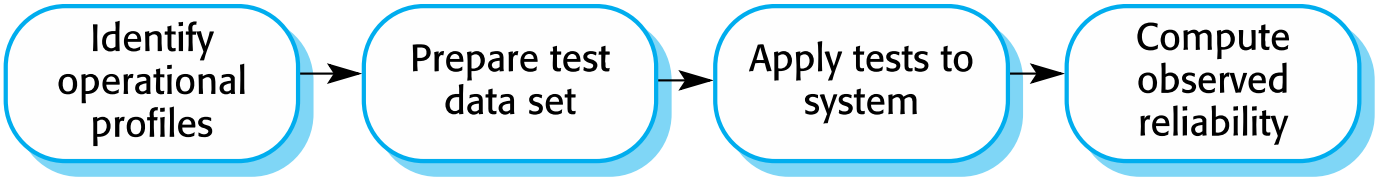
Statistical testing



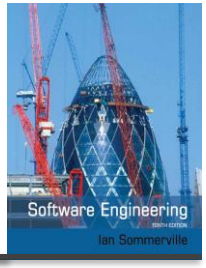
- Testing software for reliability rather than fault detection.
- Measuring the number of errors allows the reliability of the software to be predicted.
- An acceptable level of reliability should be specified and the software tested and amended until that level of reliability is reached.

Reliability measurement



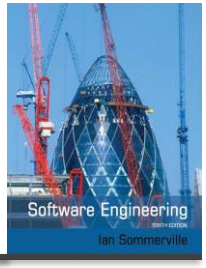


Reliability measurement problems



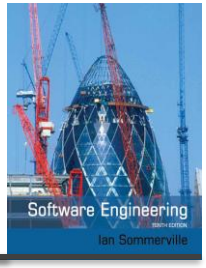
- **Operational profile uncertainty**
  - The operational profile may not be an accurate reflection of the real use of the system.
- **High costs of test data generation**
  - Costs can be very high if the test data for the system cannot be generated automatically.
- **Statistical uncertainty**
  - A statistically significant number of failures to compute the reliability is needed but highly reliable systems will rarely fail.
- **Recognizing failure**
  - It is not always obvious when a failure has occurred as there may be conflicting interpretations of a specification.

Key points



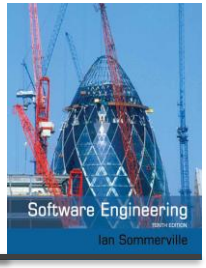
- Software reliability can be achieved by avoiding the introduction of faults, by detecting and removing faults before system deployment and by including fault tolerance facilities that allow the system to remain operational after a fault has caused a system failure.
- Reliability requirements can be defined quantitatively in the system requirements specification.
- Reliability metrics include probability of failure on demand (POFOD), rate of occurrence of failure (ROCOF) and availability (AVAIL).

Key points

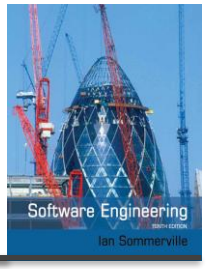


- **Functional reliability requirements** are requirements for system functionality, such as checking and redundancy requirements, which help the system meet its non-functional reliability requirements.
- **Dependable system architectures** are system architectures that are designed for fault tolerance.
- There are a **number of architectural styles that support fault tolerance** including protection systems, self-monitoring architectures and N-version programming.

Key points

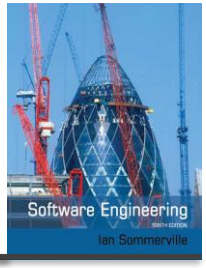


- **Software diversity** is difficult to achieve because it is practically impossible to ensure that each version of the software is truly independent.
- **Dependable programming** relies on including redundancy in a program as checks on the validity of inputs and the values of program variables.
- **Statistical testing** is used to estimate software reliability. It relies on testing the system with test data that matches an operational profile, which reflects the distribution of inputs to the software when it is in use.



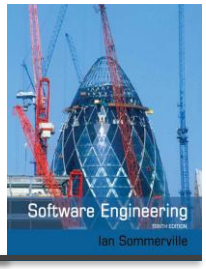
Chapter 13 – Security Engineering

Topics covered



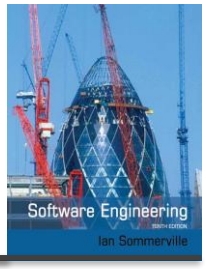
- ✧ Security and dependability
- ✧ Security and organizations
- ✧ Security requirements
- ✧ Secure systems design
- ✧ Security testing and assurance

Security engineering



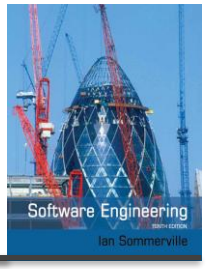
- **Security engineering**
  - Tools, techniques and methods to support the development and maintenance of systems that can resist malicious attacks that are intended to damage a computer-based system or its data.
- A sub-field of the broader field of computer security.

Security dimensions



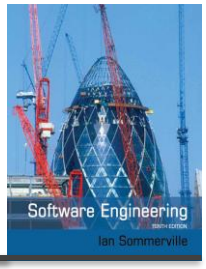
- ***Confidentiality***
  - Information in a system may be disclosed or made accessible to people or programs that are not authorized to have access to that information.
- ***Integrity***
  - Information in a system may be damaged or corrupted making it unusual or unreliable.
- ***Availability***
  - Access to a system or its data that is normally available may not be possible.

Security levels



- **Infrastructure security**, which is concerned with maintaining the security of all systems and networks that provide an infrastructure and a set of shared services to the organization.
- **Application security**, which is concerned with the security of individual application systems or related groups of systems.
- **Operational security**, which is concerned with the secure operation and use of the organization's systems.

System layers where security may be compromised



Application

Reusable components and libraries

Middleware

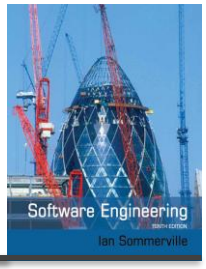
Database management

Generic, shared applications (browsers, e--mail, etc)

Operating System

Network Computer hardware

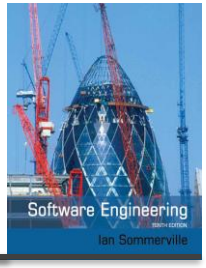
Application/infrastructure security



- **Application security**
  - a software engineering problem
  - system is designed to resist attacks.
- **Infrastructure security** is
  - a systems management problem
  - infrastructure is configured to resist attacks.

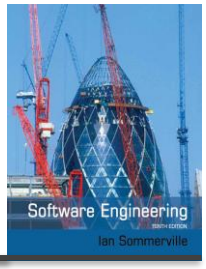
- The focus is application security rather than infrastructure security.

System security management

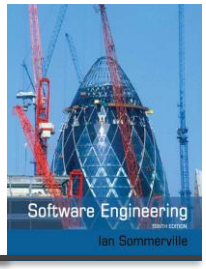


- **User and permission management**
  - Adding and removing users from the system and setting up appropriate permissions for users
- **Software deployment and maintenance**
  - Installing application software and middleware and configuring these systems so that vulnerabilities are avoided.
- **Attack monitoring, detection and recovery**
  - Monitoring the system for unauthorized access, design strategies for resisting attacks and develop backup and recovery strategies.

Operational security

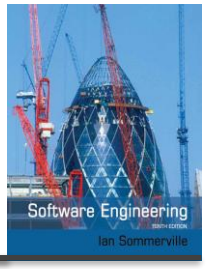


- Primarily a human and social issue
- Ensuring the people do not take actions that may compromise system security
  - E.g. Tell others passwords, leave computers logged on
- Users sometimes take insecure actions to make it easier for them to do their jobs
  - a **trade-off between system security and system effectiveness**.



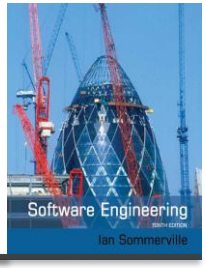
Security and dependability

Security



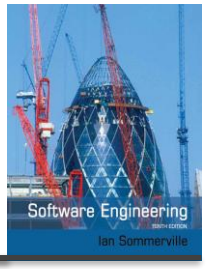
- The **security of a system** is a system property that reflects the system's ability to protect itself from accidental or deliberate external attack.
  - Is an essential pre-requisite for availability, reliability and safety.

Security terminology



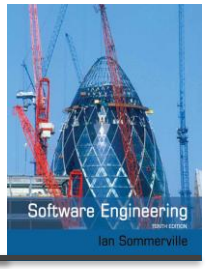
Term	Definition
Asset	Something of value which has to be protected. The asset may be the **software system** itself or **data** used by that system.
Attack	An exploitation of a system's vulnerability. Generally, this is from outside the system and is a deliberate attempt to cause some damage.
Control	A protective measure that reduces a system's vulnerability. **Encryption** is an example of a control that reduces a vulnerability of a weak access control system
Exposure	Possible loss or harm to a computing system. This can be loss or damage to data, or can be a loss of time and effort if recovery is necessary after a security breach.
Threat	Circumstances that have potential to cause loss or harm. You can think of these as a system vulnerability that is subjected to an attack.
Vulnerability	A weakness in a computer-based system that may be exploited to cause loss or harm.

Threat types



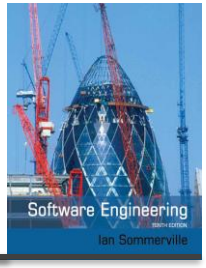
- **Interception threats** that allow an attacker to gain access to an asset.
  - A possible threat to the Mentcare system might be a situation where an attacker gains access to the records of an individual patient.
- **Interruption threats** that allow an attacker to make part of the system unavailable.
  - A possible threat might be a denial of service attack on a system database server so that database connections become impossible.

Threat types



- **Modification threats** that allow an attacker to tamper with a system asset.
  - In the Mentcare system, a modification threat would be where an attacker alters or destroys a patient record.
- **Fabrication threats** that allow an attacker to insert false information into a system.
  - This is perhaps not a credible threat in the Mentcare system but would be a threat in a banking system, where false transactions might be added to the system that transfer money to the perpetrator's bank account.

Security assurance



✧ Vulnerability avoidance

- The system is designed so that vulnerabilities do not occur.
- For example, if there is no external network connection then external attack is impossible

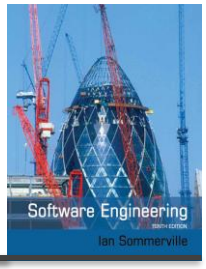
✧ Attack detection and elimination

- The system is designed so that attacks on vulnerabilities are detected and neutralised before they result in an exposure.
- For example, virus checkers find and remove viruses before they infect a system

✧ Exposure limitation and recovery

- The system is designed so that the adverse consequences of a successful attack are minimised.
- For example, a backup policy allows damaged information to be restored

Security and dependability



✧ *Security and reliability*

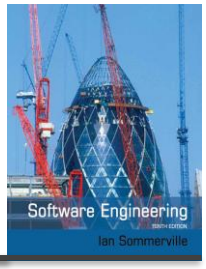
- If a system is attacked and the system or its data are corrupted as a consequence of that attack, then this may induce system failures that compromise the reliability of the system.

✧ *Security and availability*

- A common attack on a web-based system is a denial of service attack, where a web server is flooded with service requests from a range of different sources. The aim of this attack is to make the system unavailable.

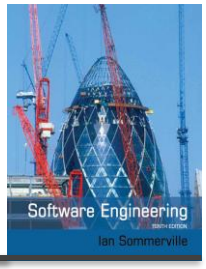
✧ *Security and resilience*

- **Resilience** is a system characteristic that reflects its ability to resist and recover from damaging events.

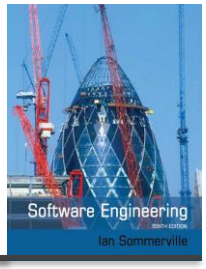


Security and organizations

Security is a business issue

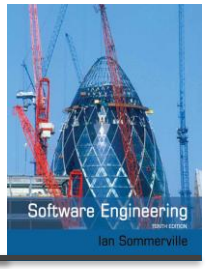


- Security is expensive
- It is important that security decisions are made in a cost-effective way
  - There is **no point in spending more than the value of an asset** to keep that asset secure.
- Organizations use a risk-based approach to support security decision making
  - a defined security policy based on security risk analysis
- Security risk analysis is a business rather than a technical process



Security requirements

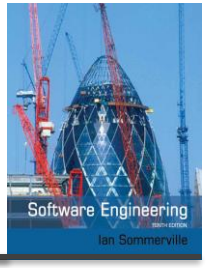
Security specification



- Security specification has something in common with safety requirements specification – in both cases, the concern is to avoid something bad happening.

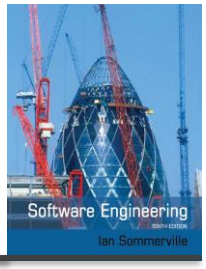
- How do they differ?

Types of security requirement



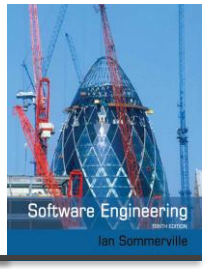
- ✧ Identification requirements.
- ✧ Authentication requirements.
- ✧ Authorisation requirements.
- ✧ Immunity requirements.
- ✧ Integrity requirements.
- ✧ Intrusion detection requirements.
- ✧ Non-repudiation requirements.
- ✧ Privacy requirements.
- ✧ Security auditing requirements.
- ✧ System maintenance security requirements.

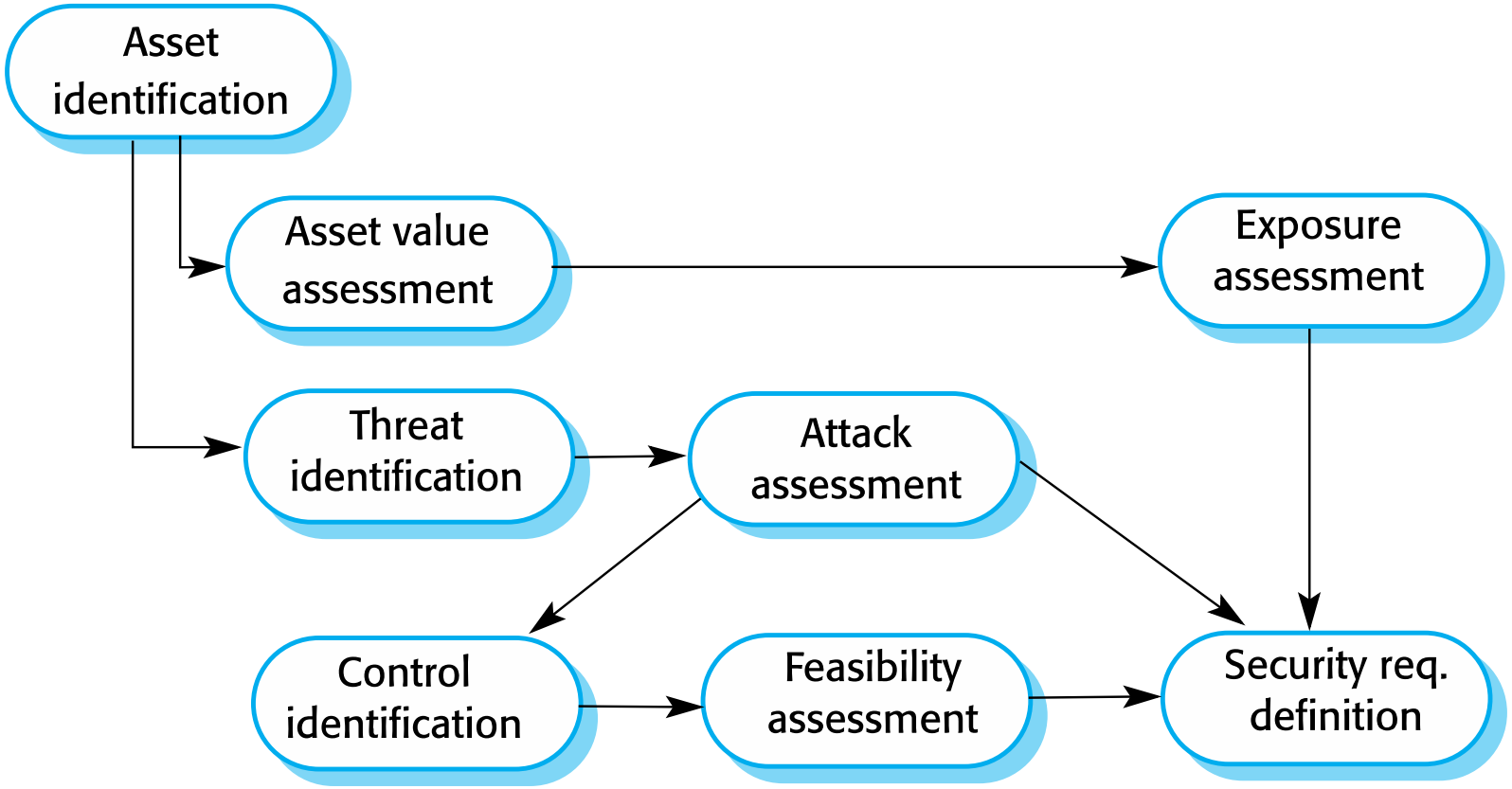
Security requirement classification



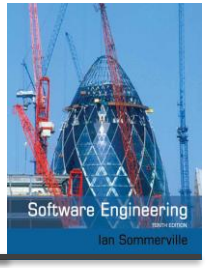
- **Risk avoidance requirements** set out the risks that should be avoided by designing the system so that these risks simply cannot arise.
- **Risk detection requirements** define mechanisms that identify the risk if it arises and neutralise the risk before losses occur.
- **Risk mitigation requirements** set out how the system should be designed so that it can recover from and restore system assets after some loss has occurred.

The preliminary risk assessment process for security requirements



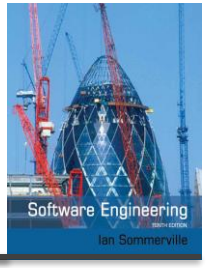


Security risk assessment



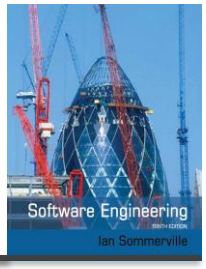
- **Asset identification**
  - Identify the key system assets (or services) that have to be protected.
- **Asset value assessment**
  - Estimate the value of the identified assets.
- **Exposure assessment**
  - Assess the potential losses associated with each asset.
- **Threat identification**
  - Identify the most probable threats to the system assets

Security risk assessment



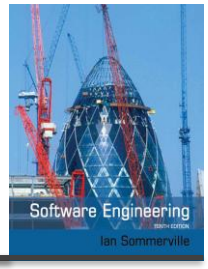
- **Attack assessment**
  - Decompose threats into possible attacks on the system and the ways that these may occur.
- **Control identification**
  - Propose the controls that may be put in place to protect an asset.
- **Feasibility assessment**
  - Assess the technical feasibility and cost of the controls.
- **Security requirements definition**
  - Define system security requirements. These can be infrastructure or application system requirements.

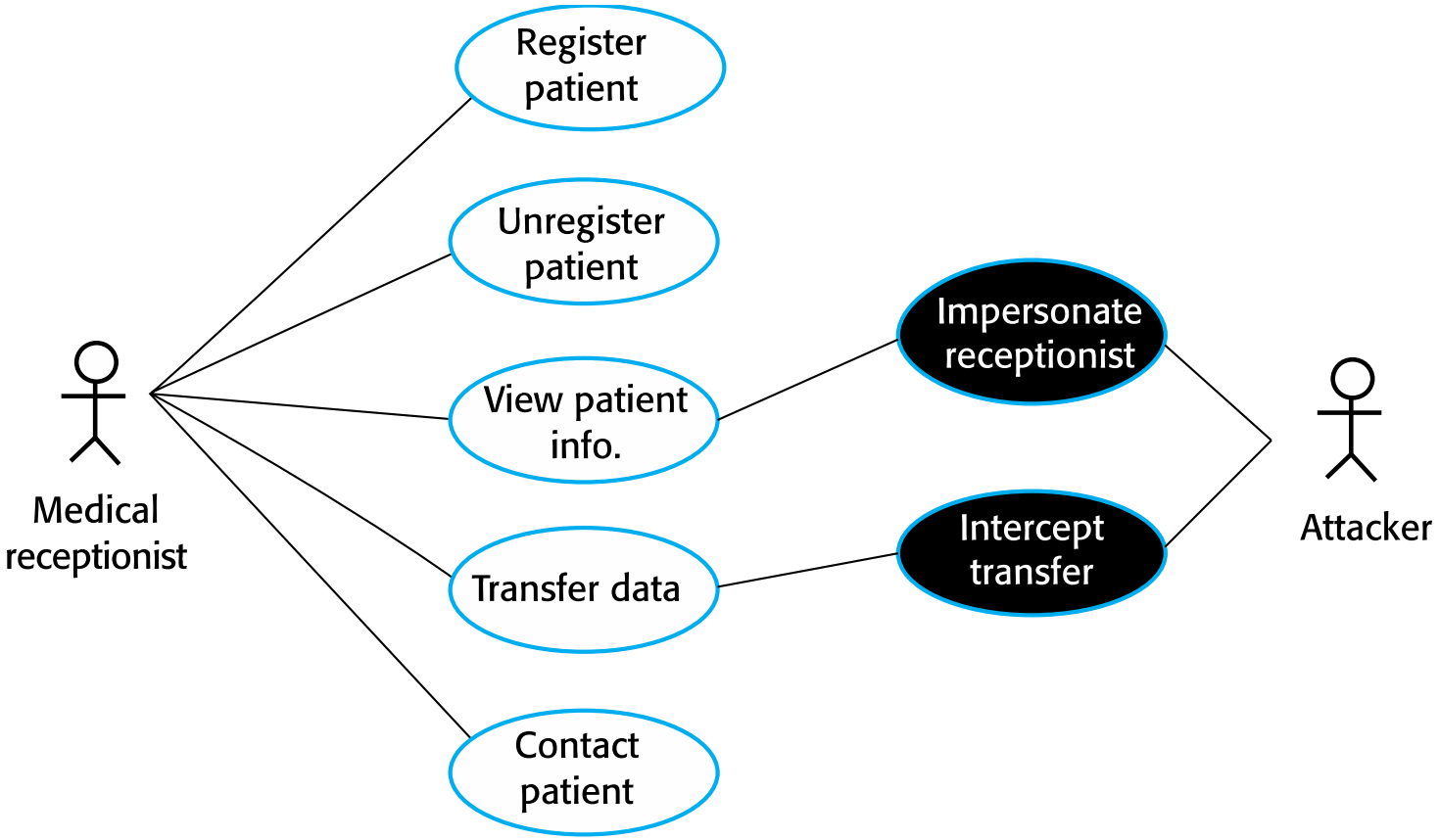
Misuse cases



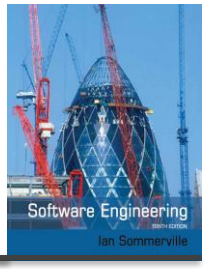
- Misuse cases are instances of threats to a system
- **Interception threats**
  - Attacker gains access to an asset
- **Interruption threats**
  - Attacker makes part of a system unavailable
- **Modification threats**
  - A system asset if tampered with
- **Fabrication threats**
  - False information is added to a system

Misuse cases



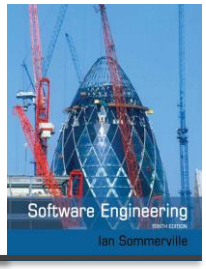


LTD: Mentcare use case – Transfer data



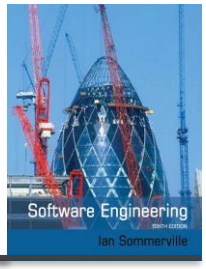
Mentcare system: Transfer data	
Actors	Medical receptionist, Patient records system (PRS)
Description	A receptionist may transfer data from the Mentcare system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary.
Stimulus	User command issued by medical receptionist.
Response	Confirmation that PRS has been updated.
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

LTD: Mentcare misuse case: Intercept transfer



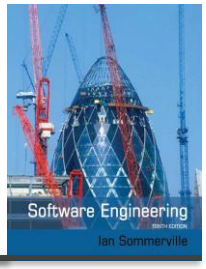
Mentcare system: Intercept transfer (Misuse case)	
Actors	Medical receptionist, Patient records system (PRS), Attacker
Description	A receptionist transfers data from his or her PC to the Mentcare system on the server. An attacker intercepts the data transfer and takes a copy of that data.
Data (assets)	Patient's personal information, treatment summary
Attacks	A network monitor is added to the system and packets from the receptionist to the server are intercepted. A spoof server is set up between the receptionist and the database server so that receptionist believes they are interacting with the real system.

LTD: Misuse case: Intercept transfer



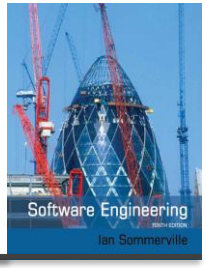
Mentcare system: Intercept transfer (Misuse case)	
Mitigations	All networking equipment must be maintained in a locked room. Engineers accessing the equipment must be accredited. All data transfers between the client and server must be encrypted. Certificate-based client-server communication must be used
Requirements	All communications between the client and the server must use the Secure Socket Layer (SSL). The https protocol uses certificate based authentication and encryption.

LTD: Organizational security policies – A Case Study (e.g Mentcare system)



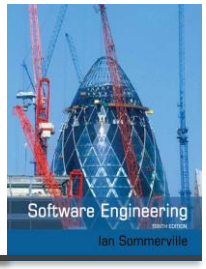
- *The assets that must be protected*
- *The level of protection that is required for different types of asset*
- *The responsibilities of individual users, managers and the organization*
- *Existing security procedures and technologies that should be maintained*
- *Risk assessment and management*
- *Risk management involves*

LTD: Mentcare system or any System



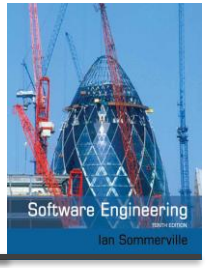
- ✧ Asset analysis in a preliminary risk assessment report for the Mentcare system
- ✧ Security requirements for the Mentcare system
- ✧ Protection requirements
- ✧ Design risk assessment

LTD:



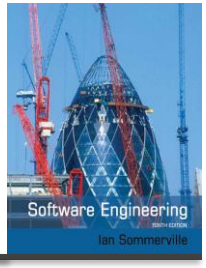
- *Considering research on the Internet and the unresolved issues on the technical complexity, uncertainty, and fluidity of authenticity, integrity, non-repudiation, and reliability of electronic document in the era of AI (be it text, voice, motion or motionless pictures) in the 21st century, kindly advise a client on the deep current and future technical issues and solutions, if any, on the use of AI applications in creating, curating, and altering electronic documents, whether hacked or lawfully communicated to the receiver or falsely created or curated by someone who wants to tender the document in court?*

LTD:

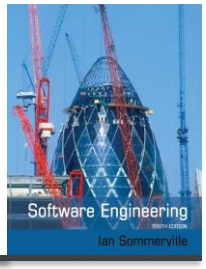


- *Put differently, some of the questions to be addressed technically include, among others:*
- *1. Can a telecommunication or internet service provider alter, change, edit or falsify the time stamp on it's system at the backend to suit the company such as MTN or anyone MTN wants to fraudulently support?*
- *2. Technically, what are the standard practice and procedure if an independent forensic investigator is legally saddled with the responsibility of auditing the system of MTN, for example, where there is a contestation on an electronic document?*

LTD:

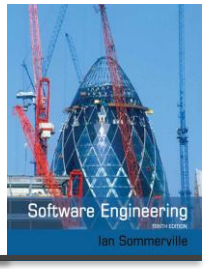


- *3. Prior to electronic documentation, it was relatively easier to determine the authenticity, reliability and non-repudiability of documents but what can be said of electronic document especially in the era of AI?*
- *4. What's the universally tested, and accepted technical standard practice and procedure for the authentication, integrity and non-repudiation of electronic document?*



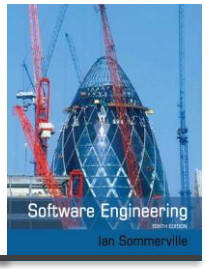
Secure systems design

Secure systems design



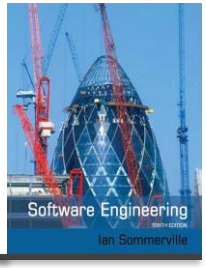
- Security should be designed into a system – it is very difficult to make an insecure system secure after it has been designed or implemented
- **Architectural design**
  - how do architectural design decisions affect the security of a system?
- **Good practice**
  - what is accepted good practice when designing secure systems?

Design compromises



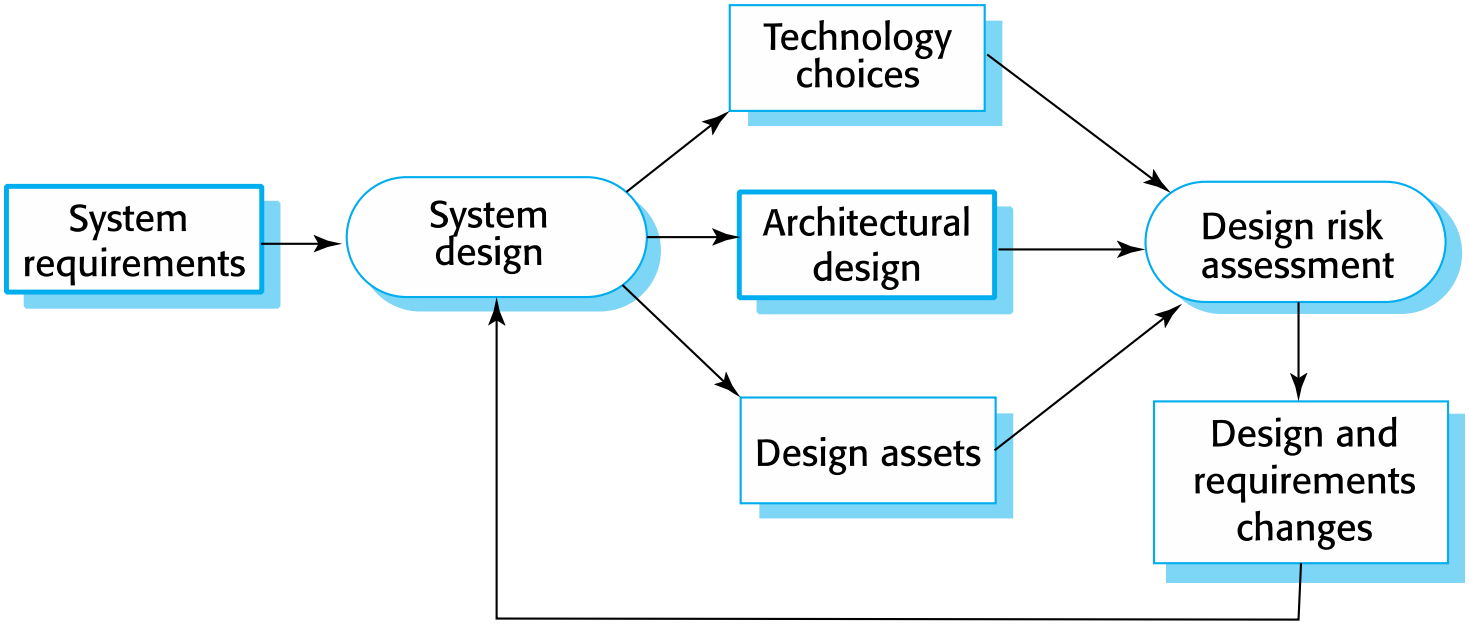
- Adding security features to a system to enhance its security affects other attributes of the system
- Performance
  - Additional security checks slow down a system so its response time or throughput may be affected
- Usability
  - Security measures may require users to remember information or require additional interactions to complete a transaction. T
  - This makes the system less usable and can frustrate system users.

Design risk assessment

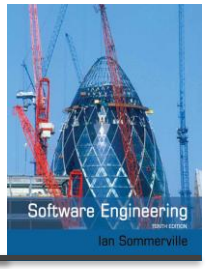


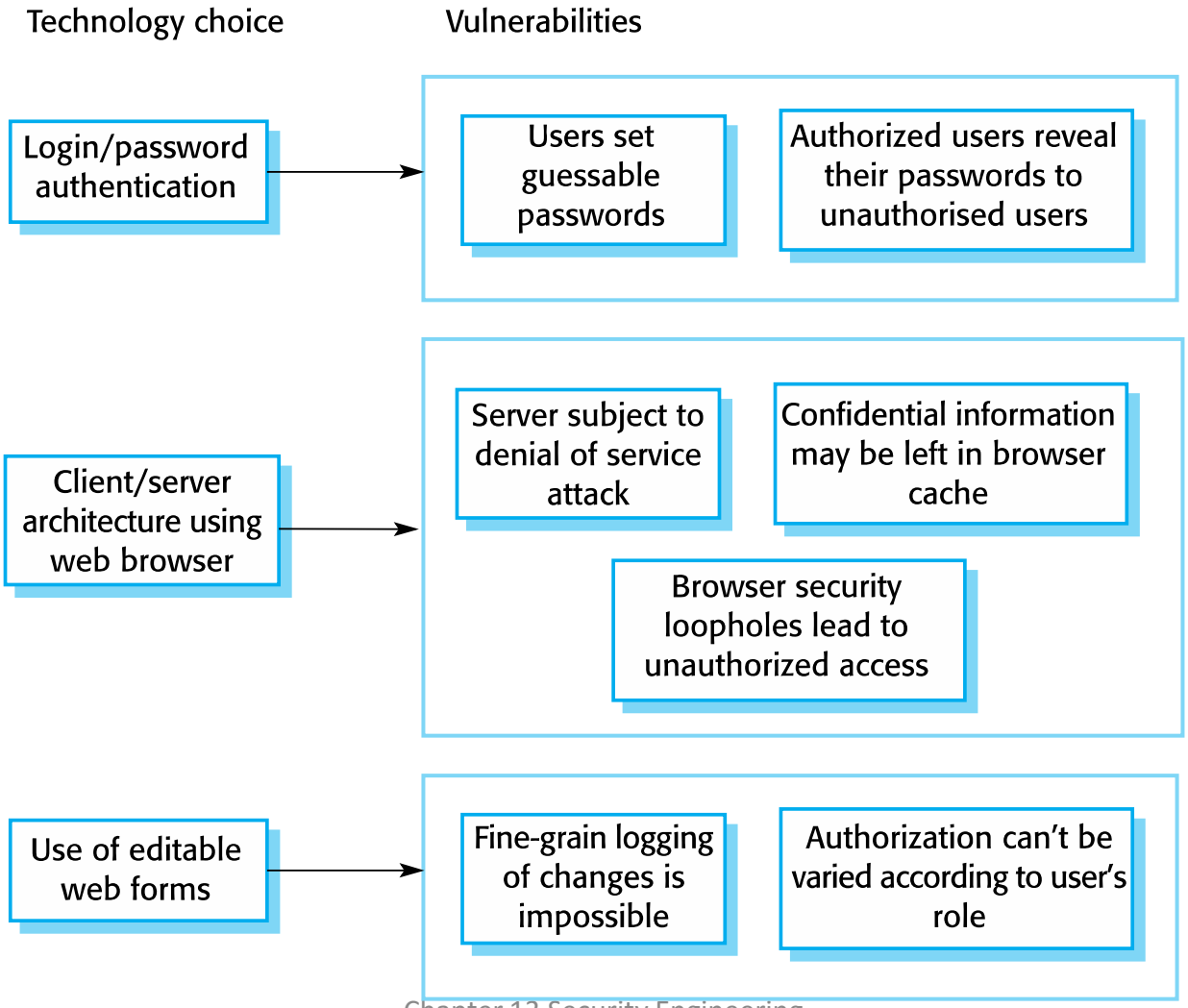
- Risk assessment while the system is being developed and after it has been deployed
- Vulnerabilities that arise from design choices may therefore be identified.

Design and risk assessment

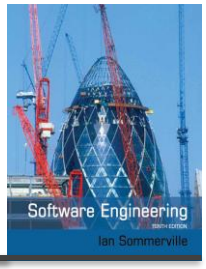


Vulnerabilities associated with technology choices



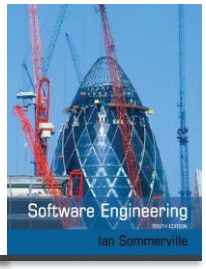


Security requirements



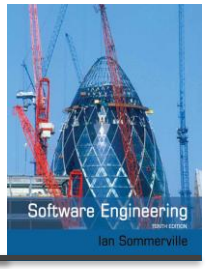
- A password checker shall be made available and shall be run daily.
- Weak passwords shall be reported to system administrators.
- Access to the system shall only be allowed by approved client computers.
- All client computers shall have a single, approved web browser installed by system administrators.

Architectural design



- **Two fundamental issues** have to be considered when designing an architecture for security.
  - Protection
    - How should the system be organised so that critical assets can be protected against external attack?
  - Distribution
    - How should system assets be distributed so that the effects of a successful attack are minimized?
- **These are potentially conflicting**
  - If assets are distributed, then they are more expensive to protect. If assets are protected, then usability and performance requirements may be compromised.

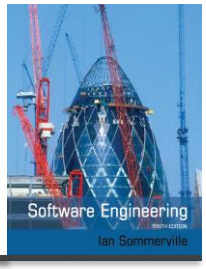
Protection

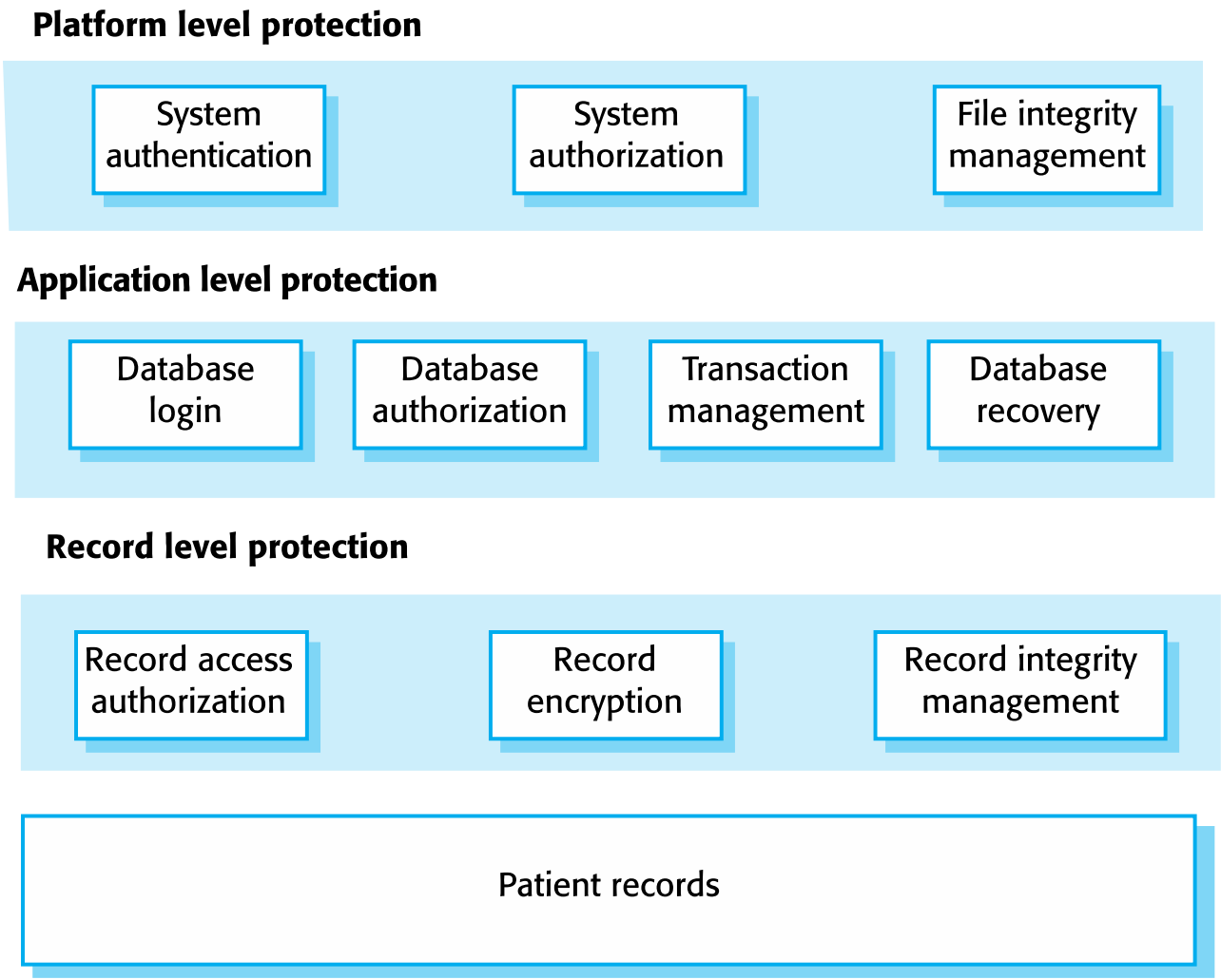


- **Platform-level protection**
  - Top-level controls on the platform on which a system runs.
- **Application-level protection**
  - Specific protection mechanisms built into the application itself e.g. additional password protection.
- **Record-level protection**
  - Protection that is invoked when access to specific information is requested

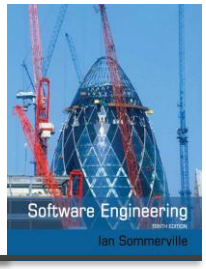
- These lead to a layered protection architecture!

A layered protection architecture

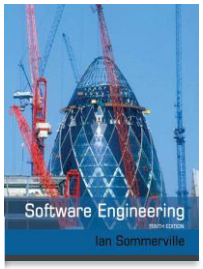


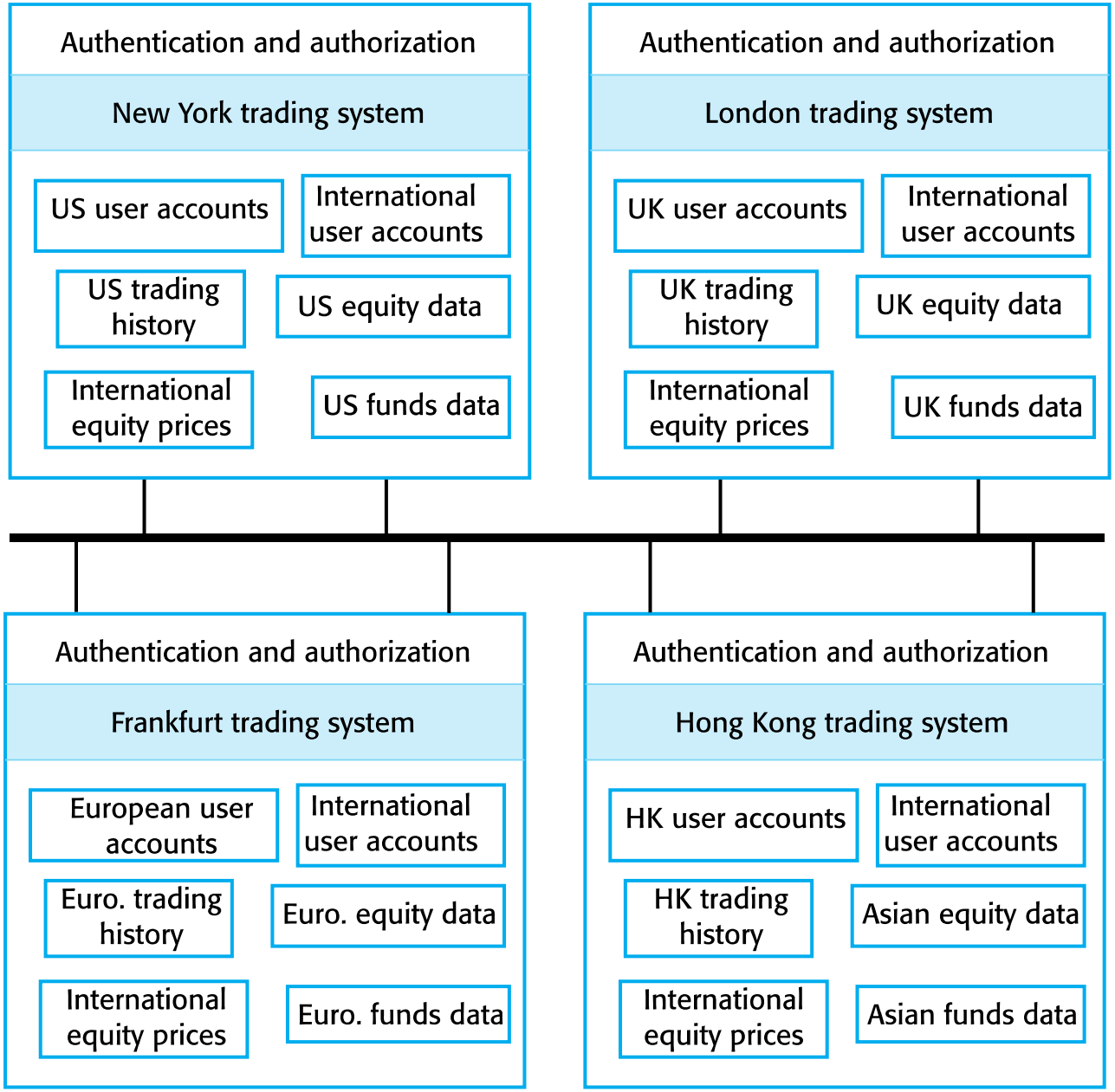


Distribution



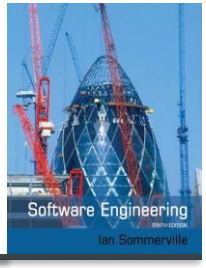
- Distributing assets
  - attacks on one system do not necessarily lead to complete loss of system service
  - Each platform has separate protection features
  - They do not share a common vulnerability
  - Important if the risk of denial of service attacks is high





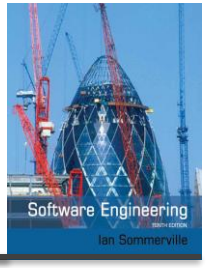
Distributed assets in an equity trading system

Design guidelines for security engineering



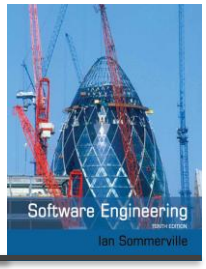
- Design guidelines encapsulate good practice in secure systems design
- Design guidelines serve two purposes:
  - They raise awareness of security issues in a software engineering team. Security is considered when design decisions are made.
  - They can be used as the basis of a review checklist that is applied during the system validation process.
- Design guidelines here are applicable during software specification and design

LTD: Design guidelines for secure systems engineering



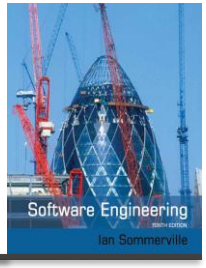
Security guidelines	
Base security decisions on an explicit security policy	
Avoid a single point of failure	
Fail securely	
Balance security and usability	
Log user actions	
Use redundancy and diversity to reduce risk	
Specify the format of all system inputs	
Compartmentalize your assets	
Design for deployment	
Design for recoverability	

Design guidelines 1-3



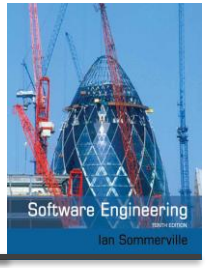
- ✧ Base decisions on an explicit security policy
  - Define a security policy for the organization that sets out the fundamental security requirements that should apply to all organizational systems.
- ✧ Avoid a single point of failure
  - Ensure that a security failure can only result when there is more than one failure in security procedures. For example, have password and question-based authentication.
- ✧ Fail securely
  - When systems fail, for whatever reason, ensure that sensitive information cannot be accessed by unauthorized users even although normal security procedures are unavailable.

Design guidelines 4-6



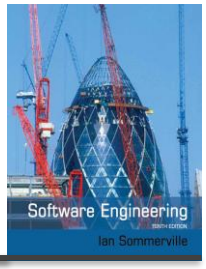
- ✧ Balance security and usability
  - Try to avoid security procedures that make the system difficult to use. Sometimes you have to accept weaker security to make the system more usable.
- ✧ Log user actions
  - Maintain a log of user actions that can be analyzed to discover who did what. If users know about such a log, they are less likely to behave in an irresponsible way.
- ✧ Use redundancy and diversity to reduce risk
  - Keep multiple copies of data and use diverse infrastructure so that an infrastructure vulnerability cannot be the single point of failure.

Design guidelines 7-10



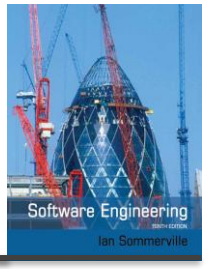
- Specify the format of all system inputs
  - If input formats are known then you can check that all inputs are within range so that unexpected inputs don't cause problems.
- Compartmentalize your assets
  - Organize the system so that assets are in separate areas and users only have access to the information that they need rather than all system information.
- Design for deployment
  - Design the system to avoid deployment problems
- Design for recoverability
  - Design the system to simplify recoverability after a successful attack.

Aspects of secure systems programming



- **Vulnerabilities are often language-specific.**
- Array bound checking is automatic in languages like Java so this is not a vulnerability that can be exploited in Java programs.
- However, millions of programs are written in C and C++ as these allow for the development of more efficient software so
  - simply avoiding the use of these languages is not a realistic option.

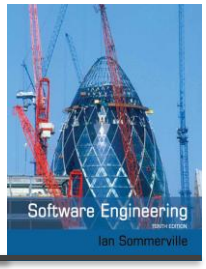
LTD: Dependable programming guidelines



Dependable programming guidelines

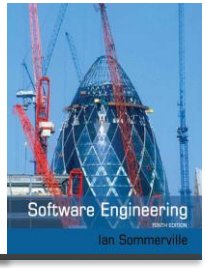
1. Limit the visibility of information in a program
2. Check all inputs for validity
3. Provide a handler for all exceptions
4. Minimize the use of error-prone constructs
5. Provide restart capabilities
6. Check array bounds
7. Include timeouts when calling external components
8. Name all constants that represent real-world values

LTD: Security testing and assurance



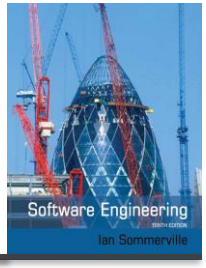
- What is security testing and assurance?
- What are problems with security testing?
- Discuss security validation

LTD: Examples of security terminology (Mentcare)



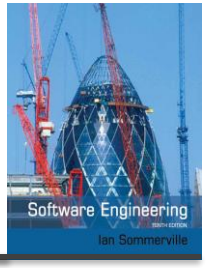
Term	Example
Asset	The records of each patient that is receiving or has received treatment.
Exposure	Potential financial loss from future patients who do not seek treatment because they do not trust the clinic to maintain their data. Financial loss from legal action by the sports star. Loss of reputation.
Vulnerability	A weak password system which makes it easy for users to set guessable passwords. User ids that are the same as names.
Attack	An impersonation of an authorized user.
Threat	An unauthorized user will gain access to the system by guessing the credentials (login name and password) of an authorized user.
Control	A password checking system that disallows user passwords that are proper names or words that are normally included in a dictionary.

Key points

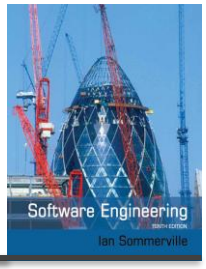


- Security engineering is concerned with how to develop systems that can resist malicious attacks
- Security threats can be threats to confidentiality, integrity or availability of a system or its data
- Security risk management is concerned with assessing possible losses from attacks and deriving security requirements to minimise losses
- To specify security requirements, you should identify the assets that are to be protected and define how security techniques and technology should be used to protect these assets.

Key points

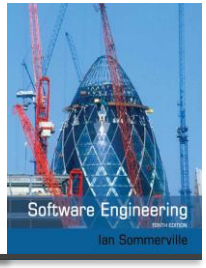


- Key issues when designing a secure systems architecture include organizing the system structure to protect key assets and distributing the system assets to minimize the losses from a successful attack.
- Security design guidelines sensitize system designers to security issues that they may not have considered. They provide a basis for creating security review checklists.
- Security validation is difficult because security requirements state what should not happen in a system, rather than what should. Furthermore, system attackers are intelligent and may have more time to probe for weaknesses than is available for security testing.



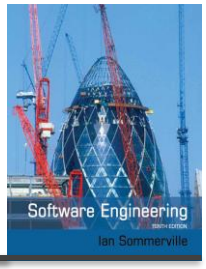
Chapter 24 - Quality Management

QUALITY CONCEPTS



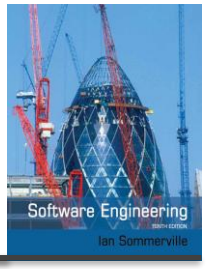
✧ Software quality & Software standards

Software quality management



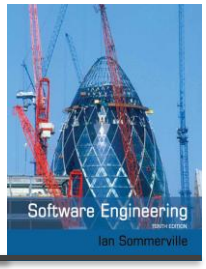
- Concerned with ensuring that the required level of quality is achieved in a software product.
- Three principal concerns:
  - At the **organizational level**, quality management is concerned with establishing a framework of organizational processes and standards that will lead to high-quality software.
  - At the **project level**, quality management involves the application of specific quality processes and checking that these planned processes have been followed.
  - At the **project level**, quality management is also concerned with establishing a quality plan for a project.

Quality management activities



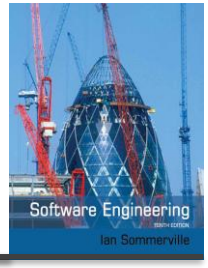
- Quality management provides an independent check on the software development process.
- The quality management process checks the project deliverables to ensure that they are consistent with organizational standards and goals
-

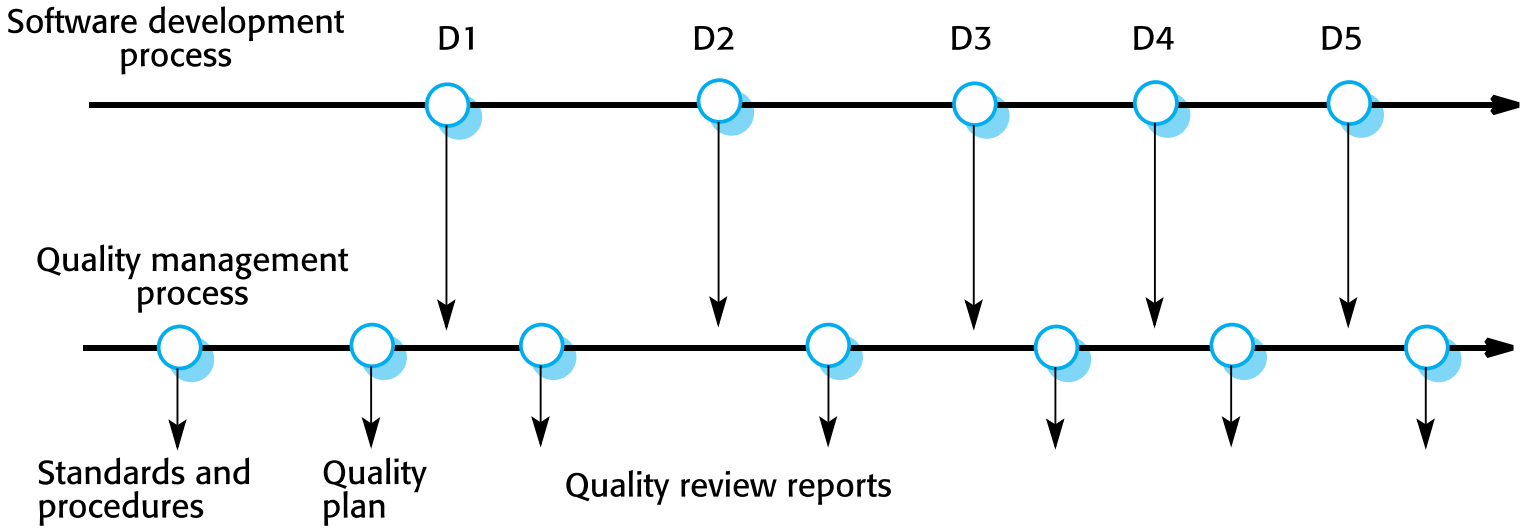
Quality management team



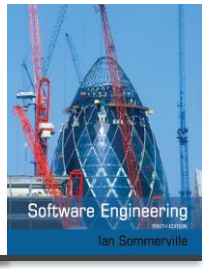
- The **quality team** should be
  - Independent from the development team so that they can take an objective view of the software.
- This allows them to report on software quality without being influenced by software development issues.

Quality management and software development



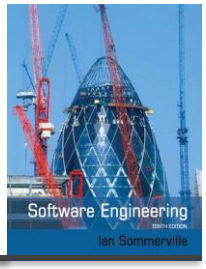


Quality planning



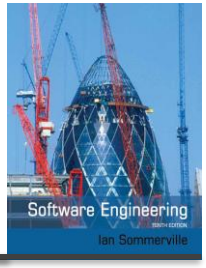
- A **quality plan**
  - sets out the desired product qualities and how these are assessed and defines the most significant quality attributes.
  - define the quality assessment process.
  - set out which organisational standards should be applied.

Quality plans



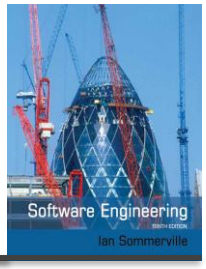
- Quality plan structure
  - Product introduction;
  - Product plans;
  - Process descriptions;
  - Quality goals;
  - Risks and risk management.
- Quality plans should be short, succinct documents

Scope of quality management



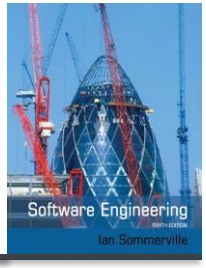
- Quality management is particularly important for large, complex systems.
- For smaller systems, quality management needs less documentation
- Establishment of "quality culture".

- **Techniques have to evolve when agile development is used**.



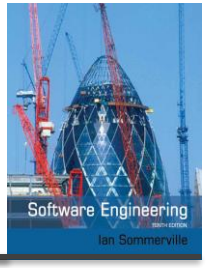
Software quality

Software quality



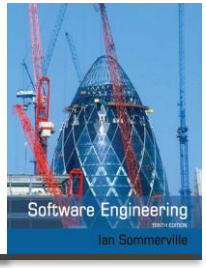
- Quality, simplistically, means that a product should meet its specification.
- This is **problematical** for software systems
  - There is a tension between **customer quality** requirements (efficiency, reliability, etc.) and **developer quality** requirements (maintainability, reusability, etc.);
  - Incomplete & inconsistent software specifications
- The focus may be 'fitness for purpose' rather than specification conformance.

Software fitness for purpose



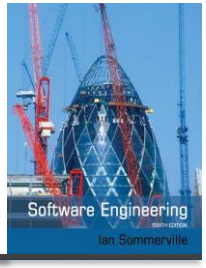
- Has the software been properly tested?
- Is the software sufficiently dependable to be put into use?
- Is the performance of the software acceptable for normal use?
- Is the software usable?
- Is the software well-structured and understandable?
- Have programming and documentation standards been followed in the development process?

Non-functional characteristics



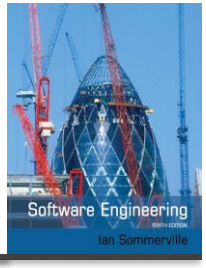
- The **subjective quality** of a software system is largely based on its non-functional characteristics.
- This reflects practical user experience.

Software quality attributes



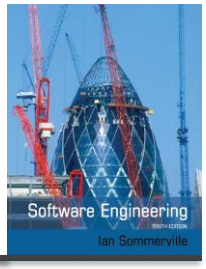
Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

Quality conflicts



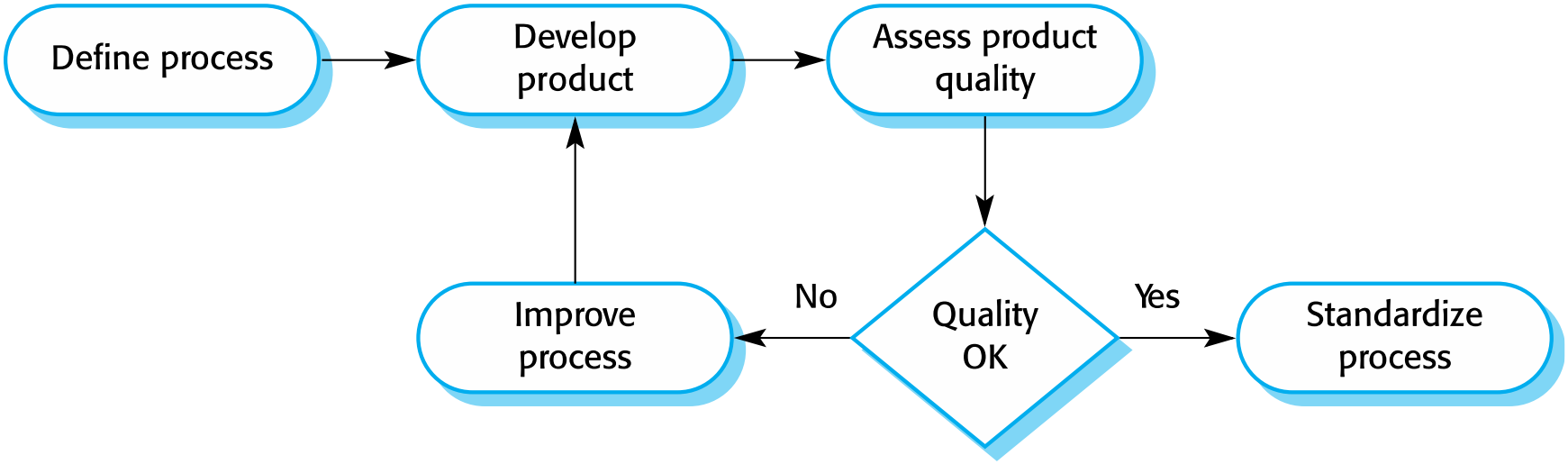
- Not possible for system to be **optimized for all attributes**
  - for example, improving robustness may lead to loss of performance.
- The **quality plan** should
  - define the most important quality attributes.
  - include a definition of the quality assessment process - maintainability or robustness

Process and product quality

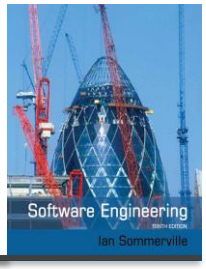


- However, there is a very complex and poorly understood relationship between **software processes** and **product quality.**
  - The application of individual skills and experience is particularly important in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.

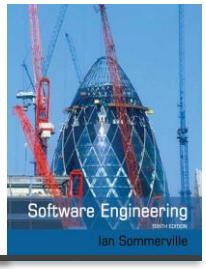
Process-based quality



Quality culture

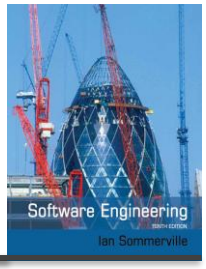


- Quality managers should aim to develop a 'quality culture':
  - everyone responsible for software development is committed to achieving a high level of product quality.
  - encourage teams to take responsibility for the quality of their work.
  - encourage professional behavior in all team members.



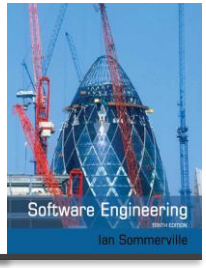
Software standards

Software standards



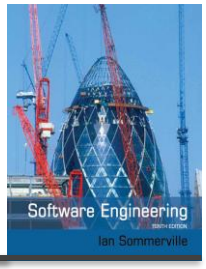
- Standards define the required attributes of a product or process.
  - They play an important role in quality management.
- Standards may be **international**, **national**, **organizational** or **project standards**.

Importance of standards



- Encapsulation of best practice - avoids repetition of past mistakes.
- They are a framework for defining what quality means in a particular setting.
- They provide continuity –
  - new staff can understand the organisation by understanding the standards that are used.

Product and process standards



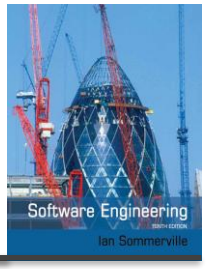
✧ *Product standards*

- Apply to the **software product** being developed.
- document standards, such as the structure of requirements documents
- coding standards, which define how a programming language should be used.

✧ *Process standards*

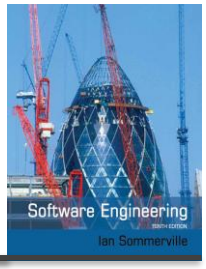
- These define the processes that should be followed during software development, which Includes:
  - definitions of specification, design and validation processes.

Product and process standards



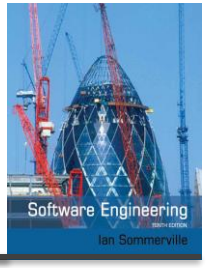
Product standards	Process standards
Design review form	Design review conduct
Requirements document structure	Submission of new code for system building
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

Problems with standards



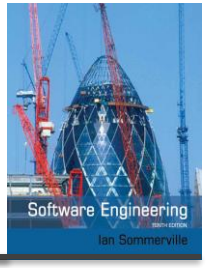
- They may not be seen as relevant and up-to-date by software engineers.
- They often involve too much bureaucratic form filling.
- If they are unsupported by software tools, tedious form filling work is often involved to maintain the documentation associated with the standards.

Standards development



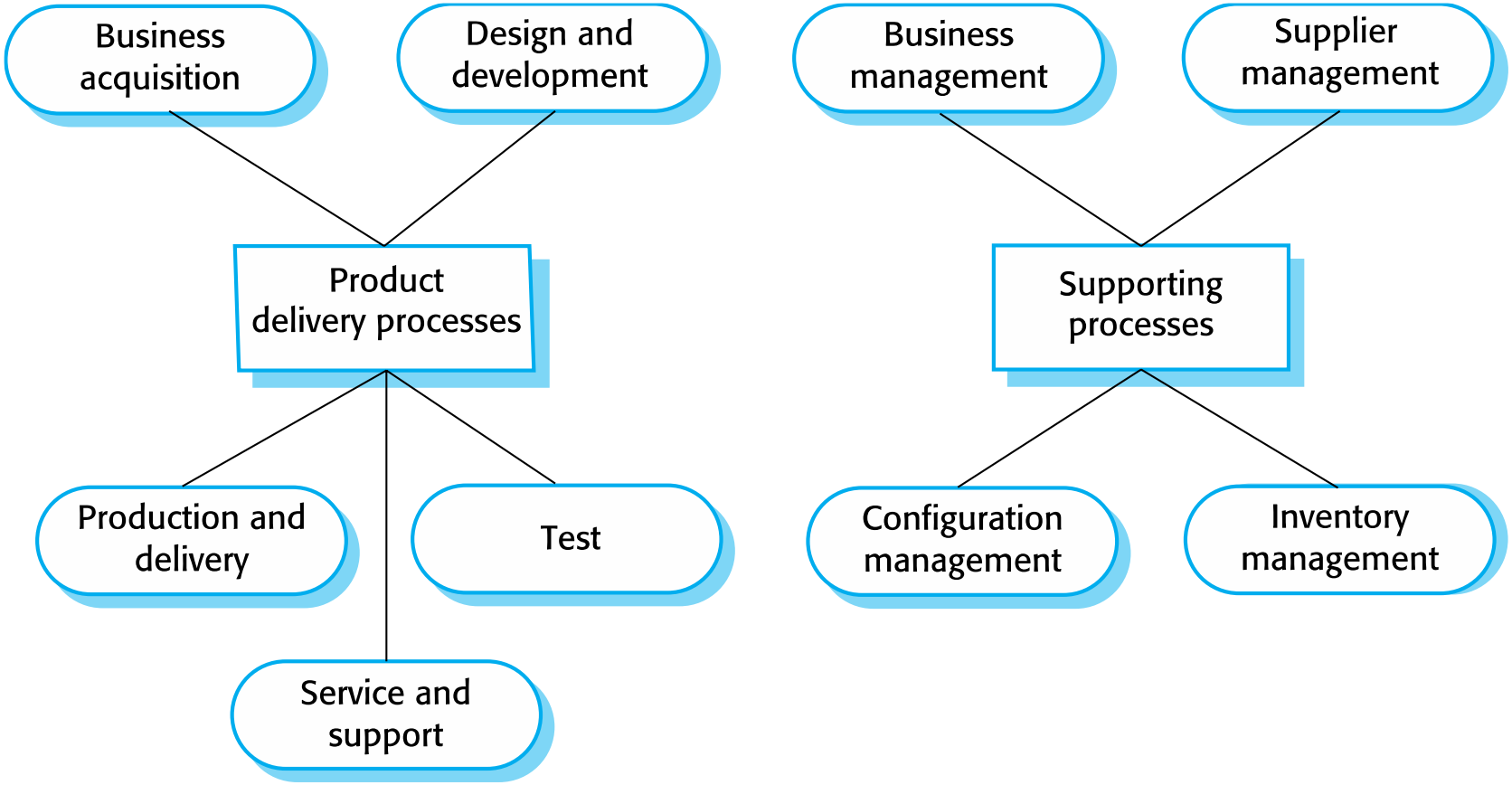
- Involve practitioners in development.
  - Engineers should understand the rationale underlying a standard.
- Review standards and their usage regularly.
  - Standards can quickly become outdated, and this reduces their credibility amongst practitioners.
- Detailed standards should have specialized tool support.
  - Excessive clerical work is the most significant complaint against standards.

ISO 9001 standards framework

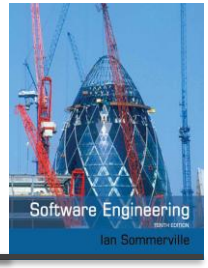


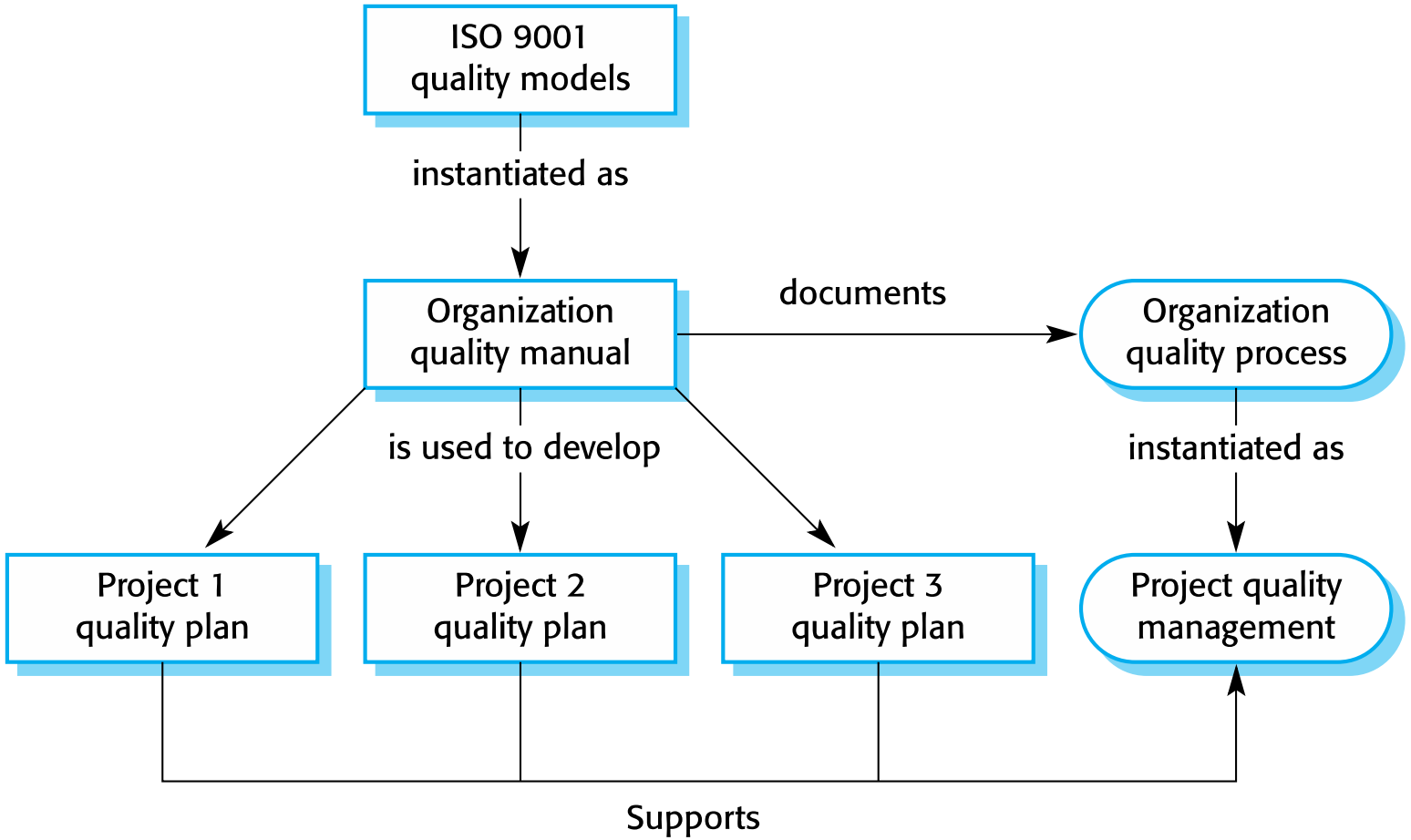
- An international set of standards that can be used as a basis for developing quality management systems.
- ISO 9001, - applies to organizations that design, develop and maintain products, including software.
- The ISO 9001 standard is a framework for developing software standards.
  - It sets out general quality principles, describes quality processes, etc.
  - should be documented in an organizational quality manual.

ISO 9001 core processes

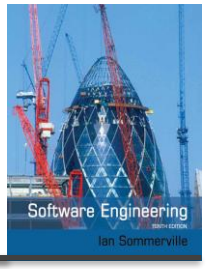


ISO 9001 and quality management



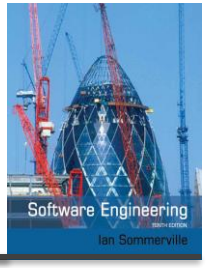


ISO 9001 certification



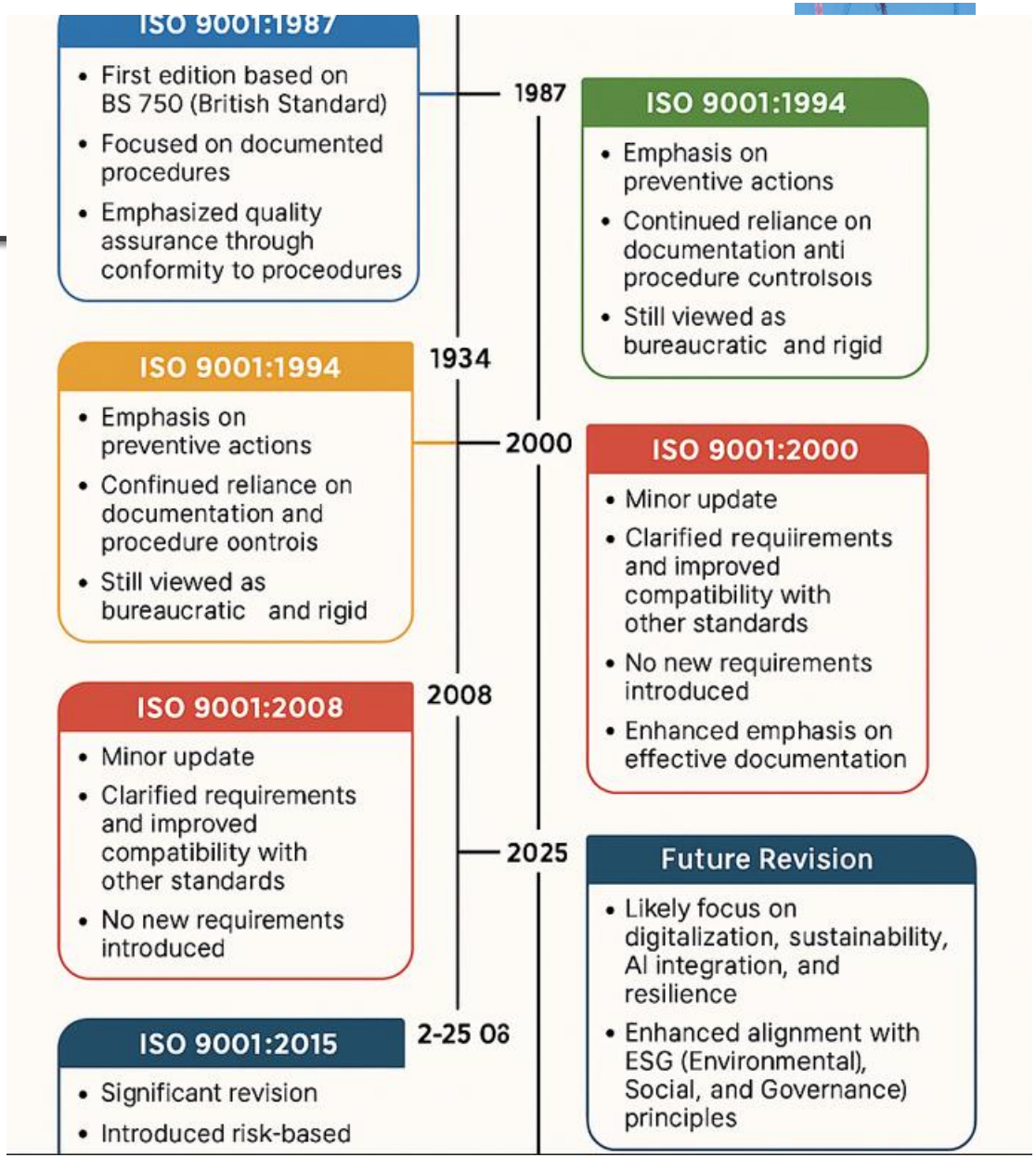
- Quality standards and procedures should be documented in an organisational quality manual.
- An external body may certify that an organisation's quality manual conforms to ISO 9000 standards.
- Some customers require suppliers to be ISO 9000 certified although the need for flexibility here is increasingly recognised.

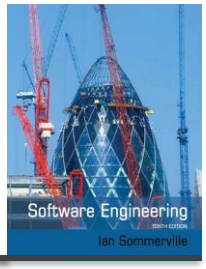
Software quality and ISO9001



- The ISO 9001 certification is inadequate because:
  - It defines quality to be the conformance to standards.
  - It takes no account of quality as experienced by users of the software.
  - standard can be met by incomplete software testing that does not include tests with different method parameters.
- So long as the defined testing procedures are followed and test records maintained, the company could be ISO 9001 certified.

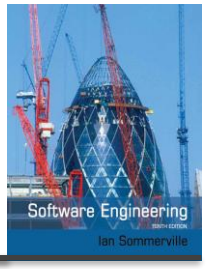
Evolution of ISO 9001





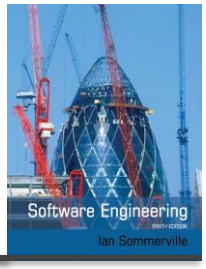
Chapter 24 - Quality Management

Review Techniques



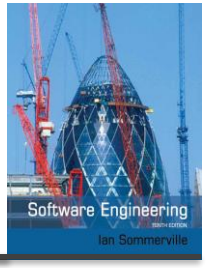
Reviews and inspections

Reviews and inspections



- A group examines part or all of a process/system and its documentation
  - to find potential problems.
- Software/documents may be **'signed off'** at a review
  - progress to the next development stage has been approved by management.

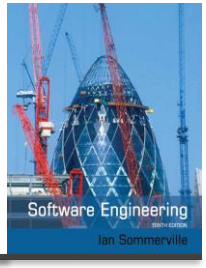
Reviews and inspections



- There are different types of review with different objectives
  - Inspections for defect removal (**product**);
  - Reviews for progress assessment (**product and process**);
  - Quality reviews (**product and standards**).

- **What can be reviewed?**
  - Code, designs, specifications, test plans, standards, etc. can all be reviewed.

Phases in the review process



✧ Pre-review activities

- Pre-review activities are concerned with review planning and review preparation

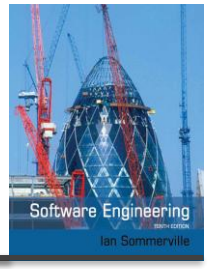
✧ The review meeting

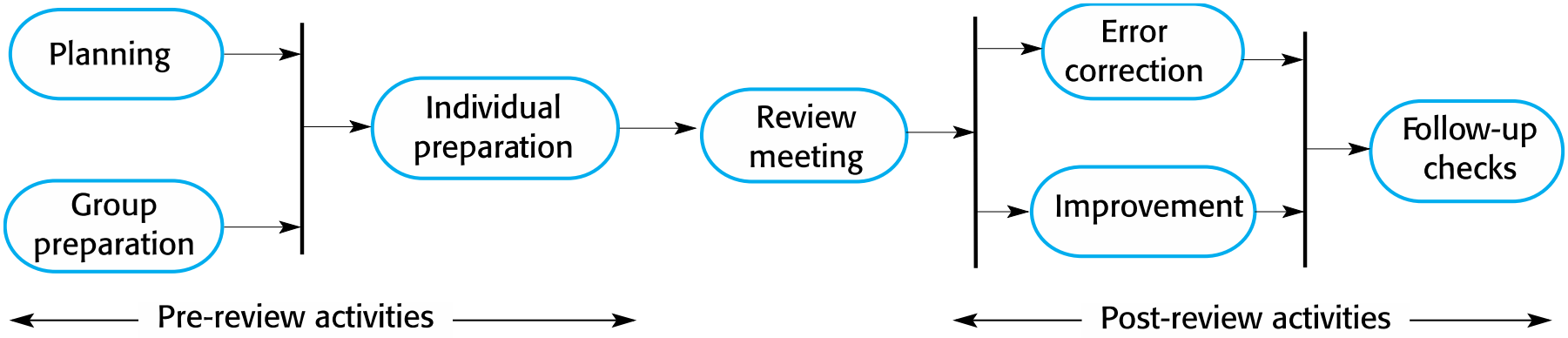
- During the review meeting, an author of the document or program being reviewed 'walk through' the document with the review team.

✧ Post-review activities

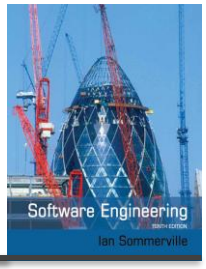
- These address the problems and issues that have been raised during the review meeting.

The software review process



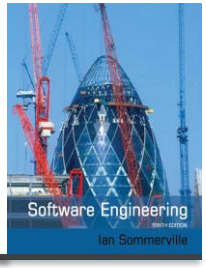


The software review process



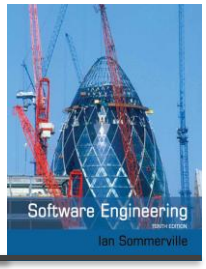
- The processes suggested for reviews
  - assume that the review team has a face-to-face meeting to discuss the software or documents that they are reviewing.

Distributed reviews



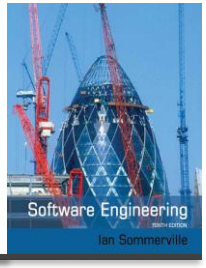
- **Distributed reviews**:
  - project teams are now often distributed,
  - sometimes across countries or continents
  - it is impractical for team members to meet face to face.
- **Remote reviewing** can be supported using shared documents where each review team member can annotate the document with their comments.

Program inspections



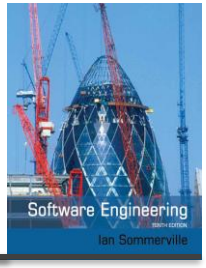
- **Peer reviews** - engineers examine the source of a system with the aim of discovering anomalies and defects.
- **Inspections**:
  - not require execution of a system so may be used before implementation.
  - They may be applied to any representation of the system (requirements, design,configuration data, test data, etc.).
  - They have been shown to be an effective technique for discovering program errors.

An inspection checklist (a)

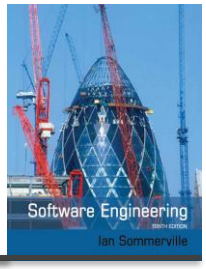


Fault class	Inspection check
Data faults	• Are all program variables initialized before their values are used? • Have all constants been named? • Should the upper bound of arrays be equal to the size of the array or Size -1? • If character strings are used, is a delimiter explicitly assigned? • Is there any possibility of buffer overflow?
Control faults	• For each conditional statement, is the condition correct? • Is each loop certain to terminate? • Are compound statements correctly bracketed? • In case statements, are all possible cases accounted for? • If a break is required after each case in case statements, has it been included?
Input/output faults	• Are all input variables used? • Are all output variables assigned a value before they are output? • Can unexpected inputs cause corruption?

An inspection checklist (b)

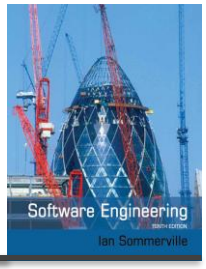


Fault class	Inspection check
Interface faults	• Do all function and method calls have the correct number of parameters? • Do formal and actual parameter types match? • Are the parameters in the right order? • If components access shared memory, do they have the same model of the shared memory structure?
Storage management faults	• If a linked structure is modified, have all links been correctly reassigned? • If dynamic storage is used, has space been allocated correctly? • Is space explicitly deallocated after it is no longer required?
Exception management faults	• Have all possible error conditions been taken into account?



Quality management and agile development

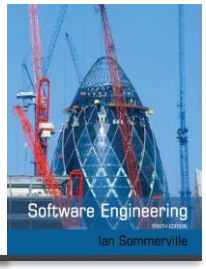
Quality management and agile development



✧ Quality management in agile development is

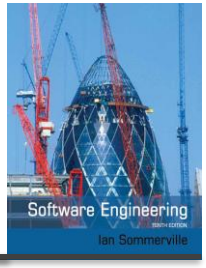
- Informal rather than document-based.
- It relies on establishing a quality culture, where all team members feel responsible for software quality.
- The agile community is fundamentally opposed to what it sees as the bureaucratic overheads of standards-based approaches and quality processes as embodied in ISO 9001.

Shared Good Practice



- ***Check before check-in***
  - Programmers are responsible for organizing their own code reviews with other team members before the code is checked in to the build system.
- ***Never break the build***
  - Team members should not check in code that causes the system to fail.
  - Developers have to test their code changes against the whole system and be confident that these work as expected.
- ***Fix problems when you see them***
  - If a programmer discovers problems in code developed by someone else, they can fix these codes directly rather than referring them back to the original developer.

Reviews and agile methods



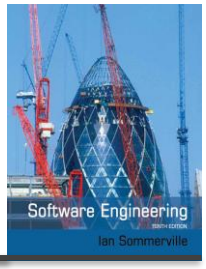
✧ In Scrum

- there is a review meeting after each iteration of the software has been completed (a sprint review), where quality issues and problems may be discussed.

✧ In Extreme Programming

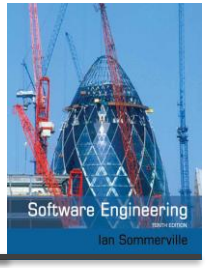
- pair programming ensures that code is constantly being examined and reviewed by another team member.

Pair programming



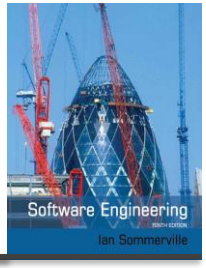
- This is an approach where 2 people are responsible for code development and work together to achieve this.
  - Code developed by an individual is therefore constantly being examined and reviewed by another team member.
  - Pair programming leads to a deep knowledge of a program, as both programmers have to understand the program in detail to continue development.

Pair programming weaknesses



- ***Mutual misunderstandings***
  - Both members of a pair may make the same mistake in understanding the system requirements.
    - Discussions may reinforce these errors.
- ***Pair reputation***
  - Pairs may be reluctant to look for errors because they do not want to slow down the progress of the project.
- ***Working relationships***
  - The pair's ability to discover defects is likely to be compromised by their close working relationship that often leads to reluctance to criticize work partners.

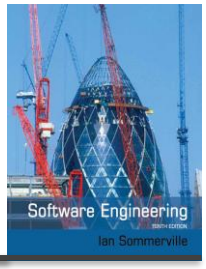
LTD: Agile QM and large systems



✧ Agile QM and large systems?

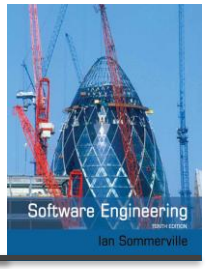
Software measurement

Software measurement



- Software measurement is concerned
  - deriving a numeric value for an attribute of a software product or process.
  - allows for objective comparisons between techniques and processes.
- Some organisations still don't make systematic use of software measurement.

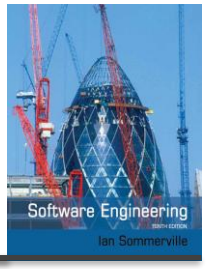
Software metric



- **Any type of measurement which relates to a software system, process or related documentation**
- Lines of code in a program, the Fog index, number of person-days required to develop a component.
- Allow the software and the software process to be quantified.
- May be used to predict product attributes or to control the software process.

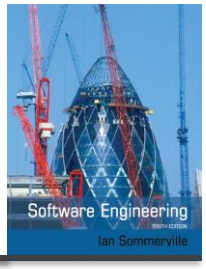
- **Product metrics** can be used for general predictions or to identify anomalous components.

LTD: Software metric



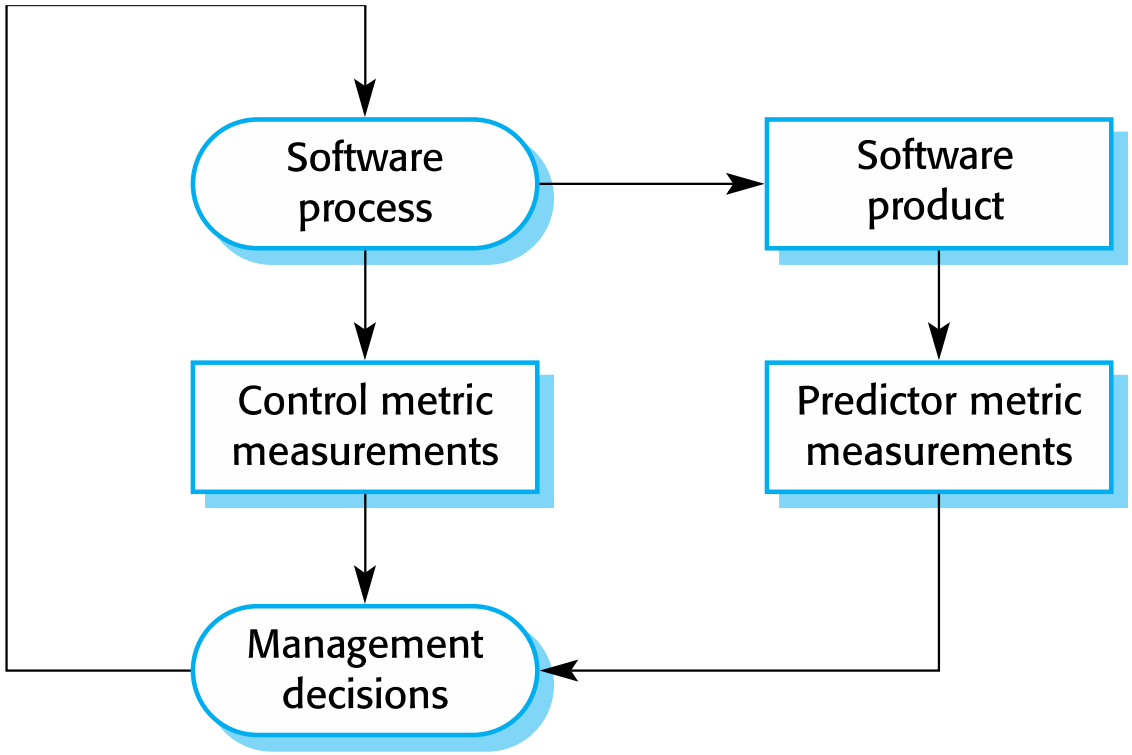
- Discuss in detail how Product metrics can be used for general predictions or to identify anomalous components.

Types of process metric

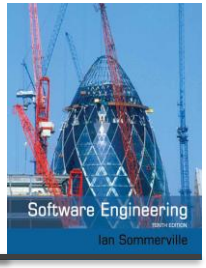


- *The time taken for a particular process to be completed*
  - This can be the total time devoted to the process, calendar time, the time spent on the process by particular engineers, and so on.
- *The resources required for a particular process*
  - Resources might include total effort in person-days, travel costs or computer resources.
- *The number of occurrences of a particular event*
  - Examples of events that might be monitored include the number of defects discovered during code inspection, the number of requirements changes requested, the number of bug reports in a delivered system and the average number of lines of code modified in response to a requirements change.

Predictor and control measurements



Use of measurements



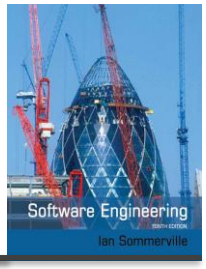
✧ To assign a value to system quality attributes

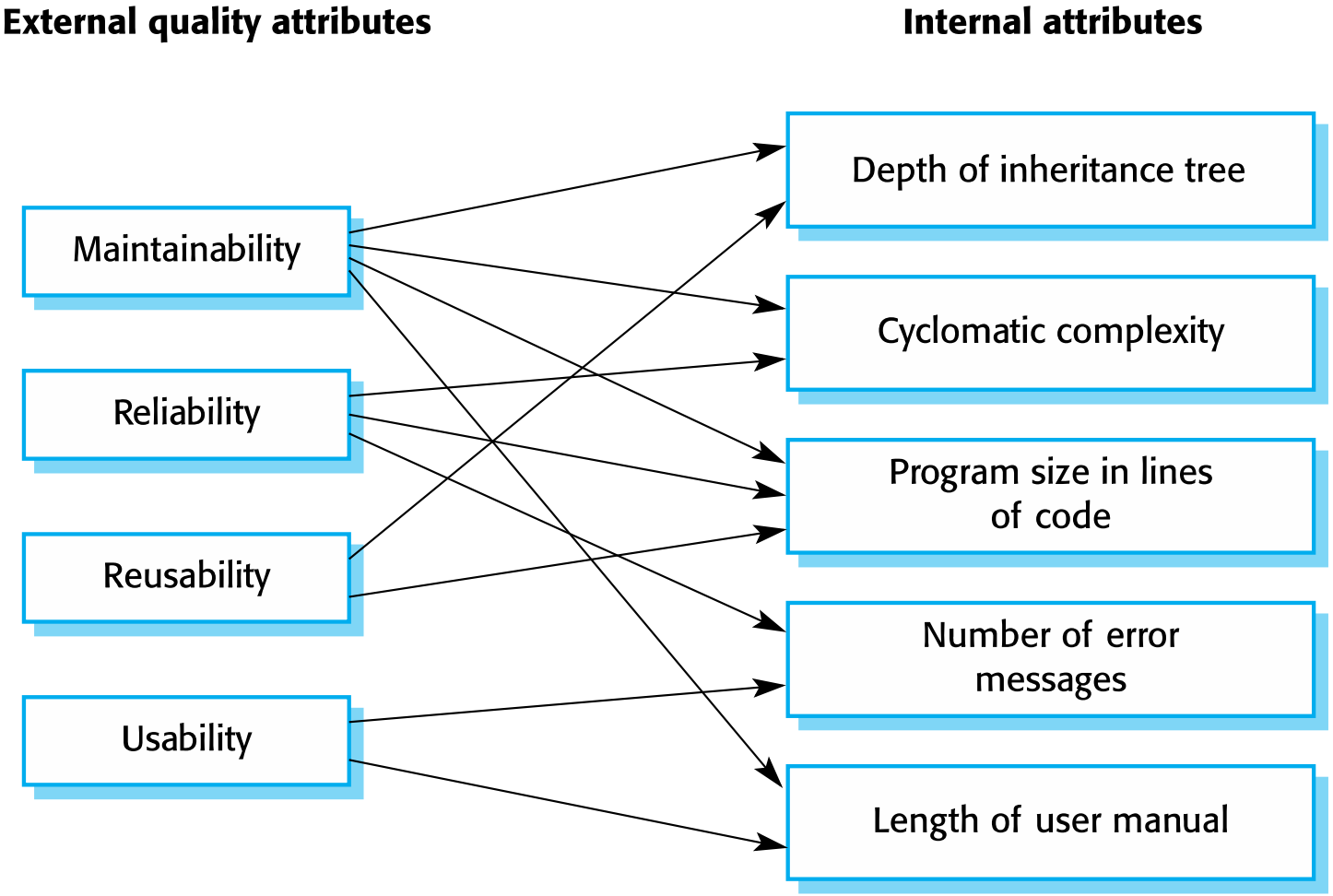
- By measuring the characteristics of system components, such as their cyclomatic complexity, and then aggregating these measurements, you can assess system quality attributes, such as **maintainability**.

✧ To identify the system components whose quality is sub-standard

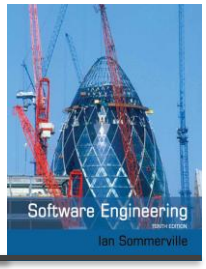
- Measurements can identify individual components with characteristics that deviate from the norm.

LTD: Relationships between internal and external software





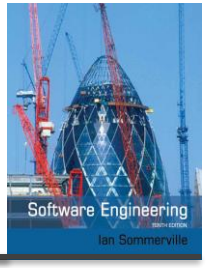
LTD: Problems with measurement in industry



- ?

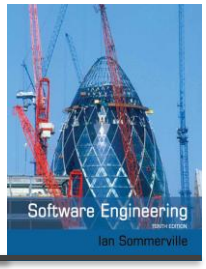
- Discuss the Problems with measurement in industry

Empirical software engineering



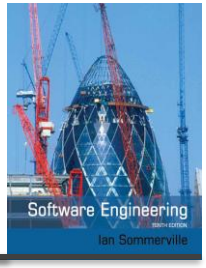
- Software measurement and metrics are the basis of empirical software engineering.
- This is a **research area** in which experiments on software systems and the collection of data about real projects has been used to form and validate hypotheses about software engineering methods and techniques.
- Research on empirical software engineering has not had a significant impact on software engineering practice.
  - It is difficult to relate generic research to a project that is different from the research study.

Product metrics



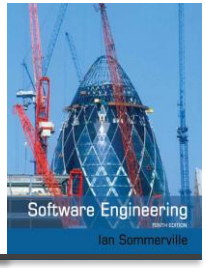
- A quality metric should be a predictor of product quality.
- Classes of product metric
  - **Dynamic metrics** which are collected by measurements made of a program in execution;
  - are closely related to software quality attributes
  - Dynamic metrics help assess efficiency and reliability
  - **Static metrics** which are collected by measurements made of the system representations;
  - an indirect relationship with quality attributes
  - Static metrics help assess complexity, understandability and maintainability.

Static software product metrics



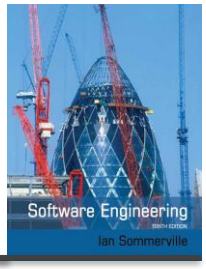
Software metric	Description
Fan-in/Fan-out	Fan-in is a measure of the number of functions or methods that call another function or method (say X). Fan-out is the number of functions that are called by function X. A high value for fan-in means that X is tightly coupled to the rest of the design and changes to X will have extensive knock-on effects. A high value for fan-out suggests that the overall complexity of X may be high because of the complexity of the control logic needed to coordinate the called components.
Length of code	This is a measure of the size of a program. Generally, the larger the size of the code of a component, the more complex and error-prone that component is likely to be. Length of code has been shown to be one of the most reliable metrics for predicting error-proneness in components.

Static software product metrics



Software metric	Description
Cyclomatic complexity	This is a measure of the control complexity of a program. This control complexity may be related to program understandability.
Length of identifiers	This is a measure of the average length of identifiers (names for variables, classes, methods, etc.) in a program. The longer the identifiers, the more likely they are to be meaningful and hence the more understandable the program.
Depth of conditional nesting	This is a measure of the depth of nesting of if-statements in a program. Deeply nested if-statements are hard to understand and potentially error-prone.
Fog index	This is a measure of the average length of words and sentences in documents. The higher the value of a document's Fog index, the more difficult the document is to understand.

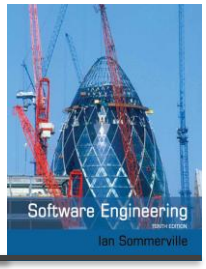
LTD: Fog index



- With illustration, discuss and calculate a Fog index using numerical examples!

- Discuss the CK object-oriented metrics suite

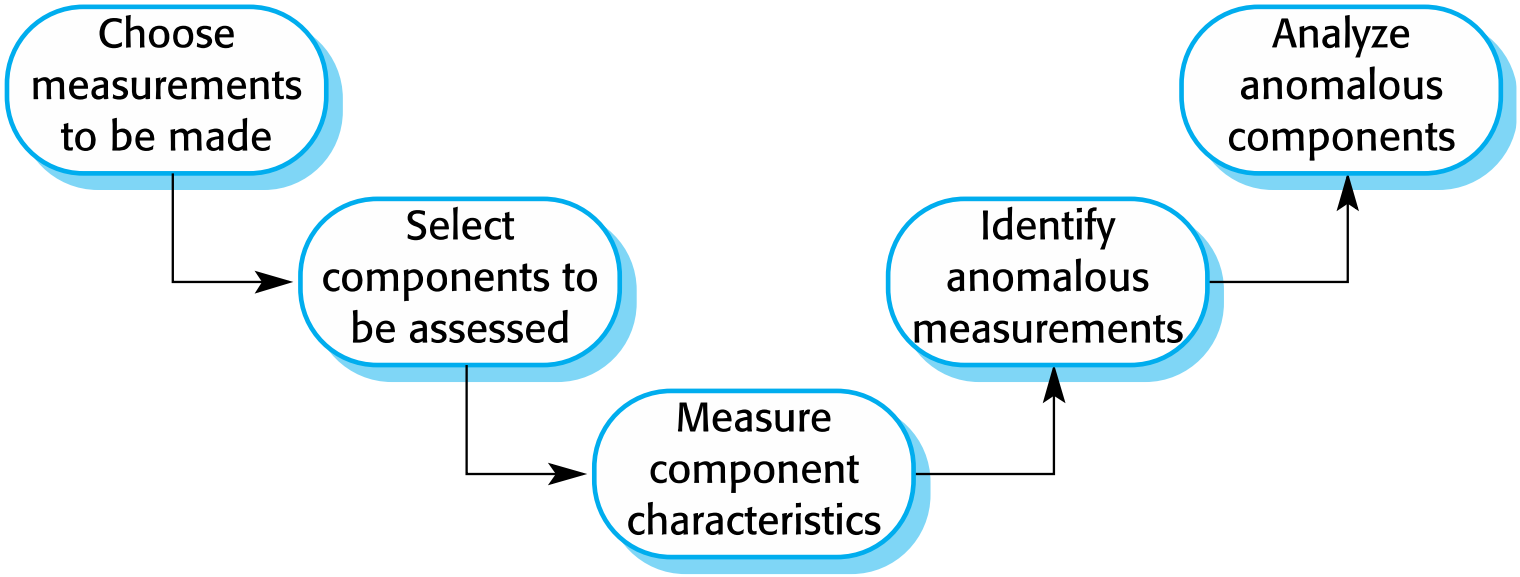
Software component analysis



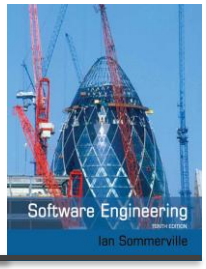
- System component can be analyzed separately using a range of metrics.

- The values of these metrics may then compared for different components and, perhaps, with
  - Also, perhaps, with historical measurement data collected on previous projects.

The process of product measurement

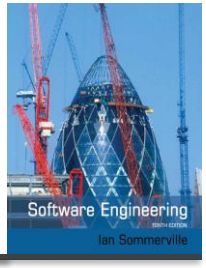


Measurement ambiguity



- When you collect quantitative data about software and software processes, you have to analyze that data to understand its meaning.
- It is easy to misinterpret data and to make inferences that are incorrect.
- You cannot simply look at the data on its own.
  - You must also consider the context where the data is collected.

LTD: Software analytics



- *Software analytics is analytics on software data for managers and software engineers with the aim of empowering software development individuals and teams to gain and share insight from their data to make better decisions.*